

CÁC CÔNG NGHỆ LẬP TRÌNH HIỆN ĐẠI

THIẾT KẾ GIAO TIẾP GIỮA CÁC DỊCH VỤ

API GATEWAYS

Các mẫu thiết kế API Gateway trong kiến trúc microservices: Gateway Routing, Gateway Aggregation và Gateway Offloading để định tuyến, tổng hợp và xử lý các tác vụ chung.

Mẫu Backends for Frontends (BFF): xây dựng backend riêng cho từng loại client (web, mobile) nhằm tối ưu hóa giao tiếp và hiệu năng.

TS. Đỗ Như Tài
Đại Học Sài Gòn
dntai@sgu.edu.vn

Microservices Communications – API Gateways

API Gateway Pattern

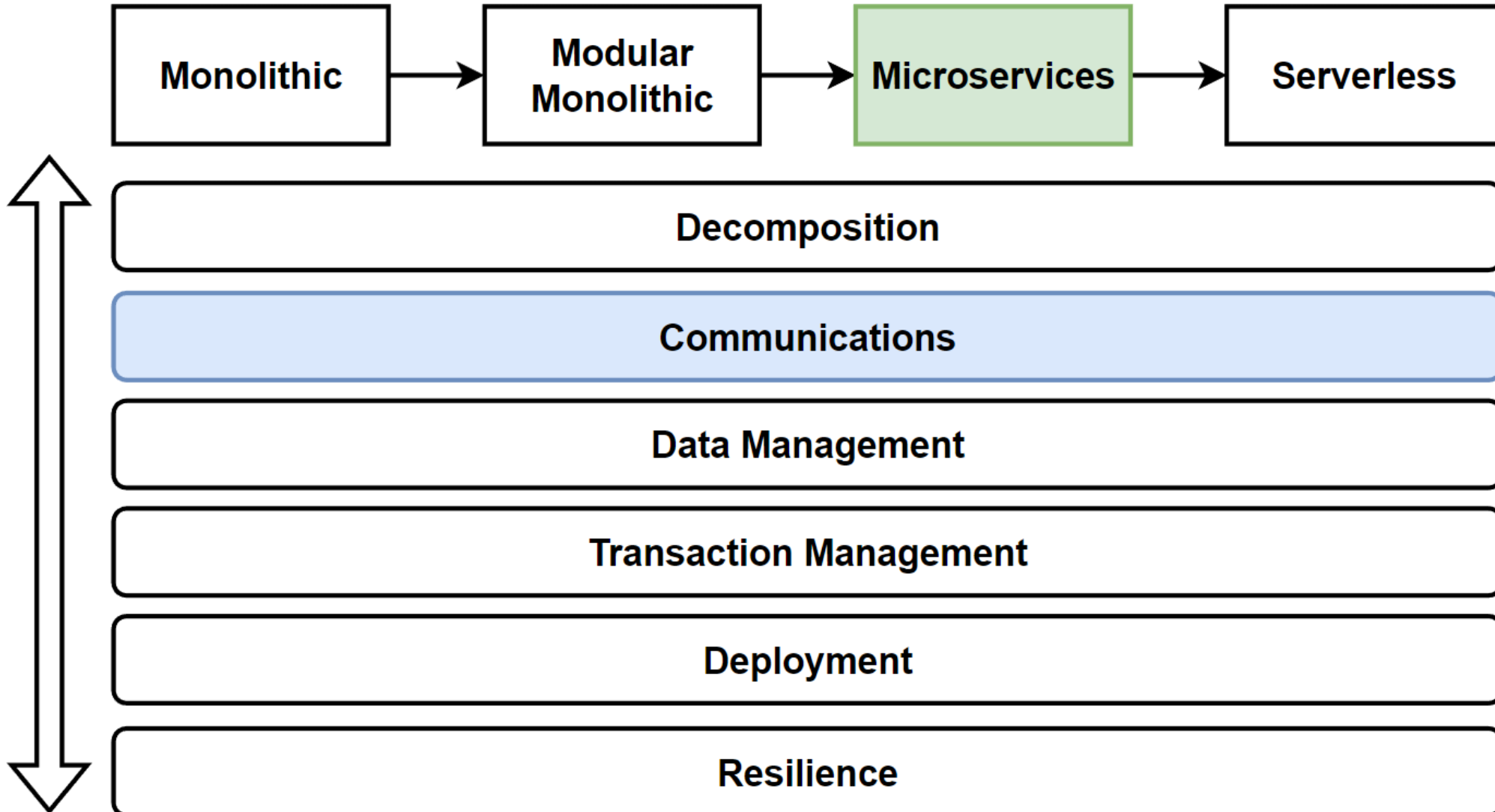
Gateway Routing Pattern

Gateway Aggregation Pattern

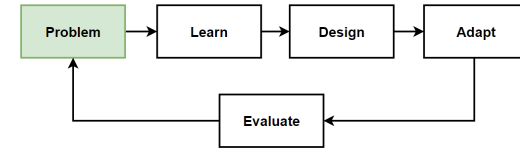
Gateway Offloading Pattern

Backends for Frontends Pattern-BFF

Architecture Design – Vertical Considerations



Problem: Direct Client-to-Service Communication



Problems

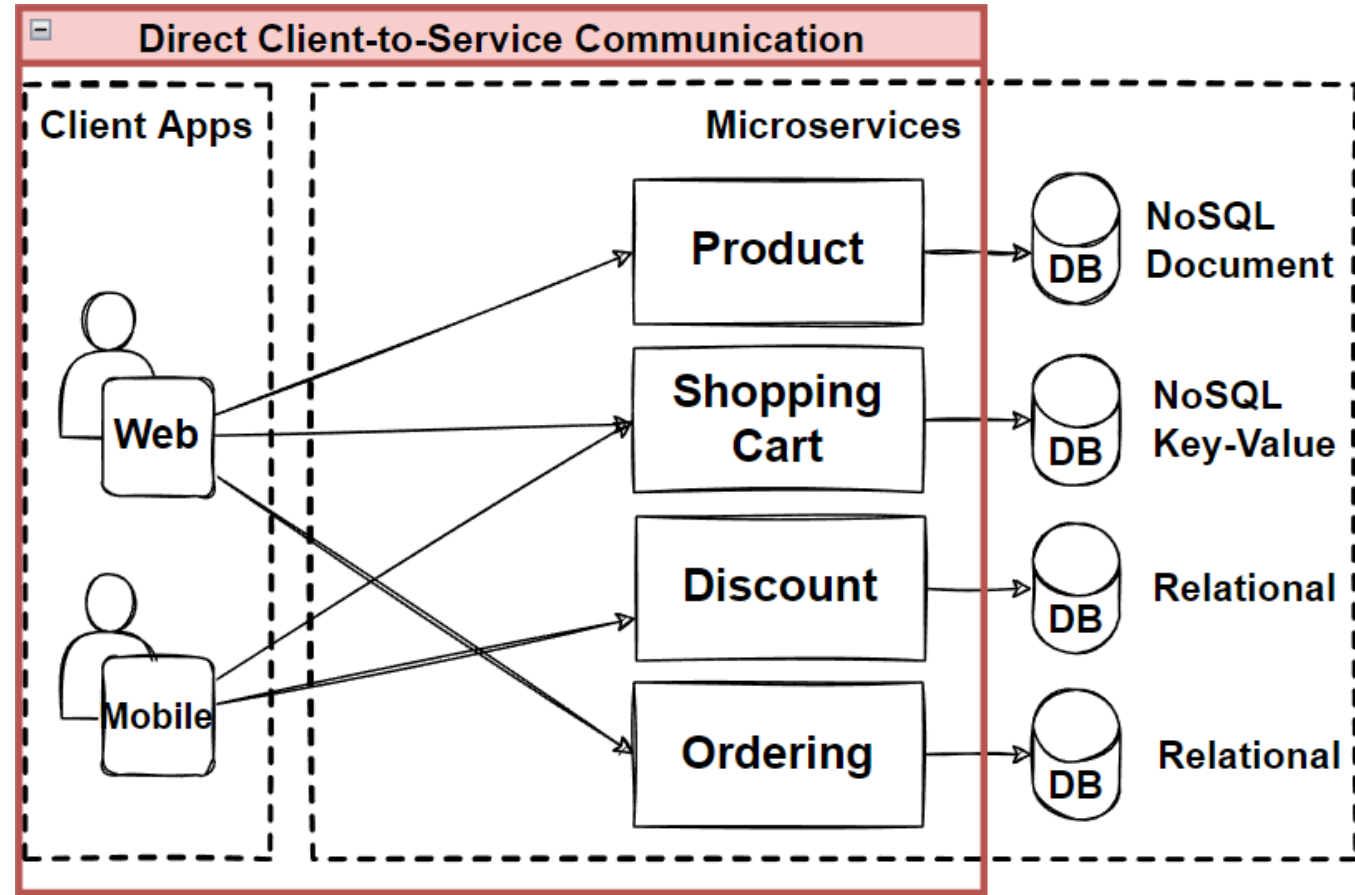
- Direct Client-to-Service Communication
- Cause to chatty calls from client to service
- Hard to manage invocations from client app with all different protocols (HTTP, GraphQL, gRPC, WebSocket)
- Increased latency and complexity on the UI side

Solutions

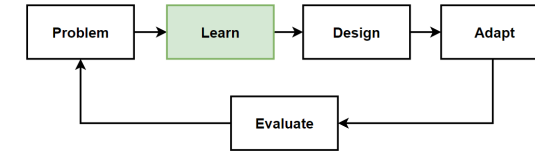
- Well-defined API Design
- **Microservices Communication Patterns**

Steps

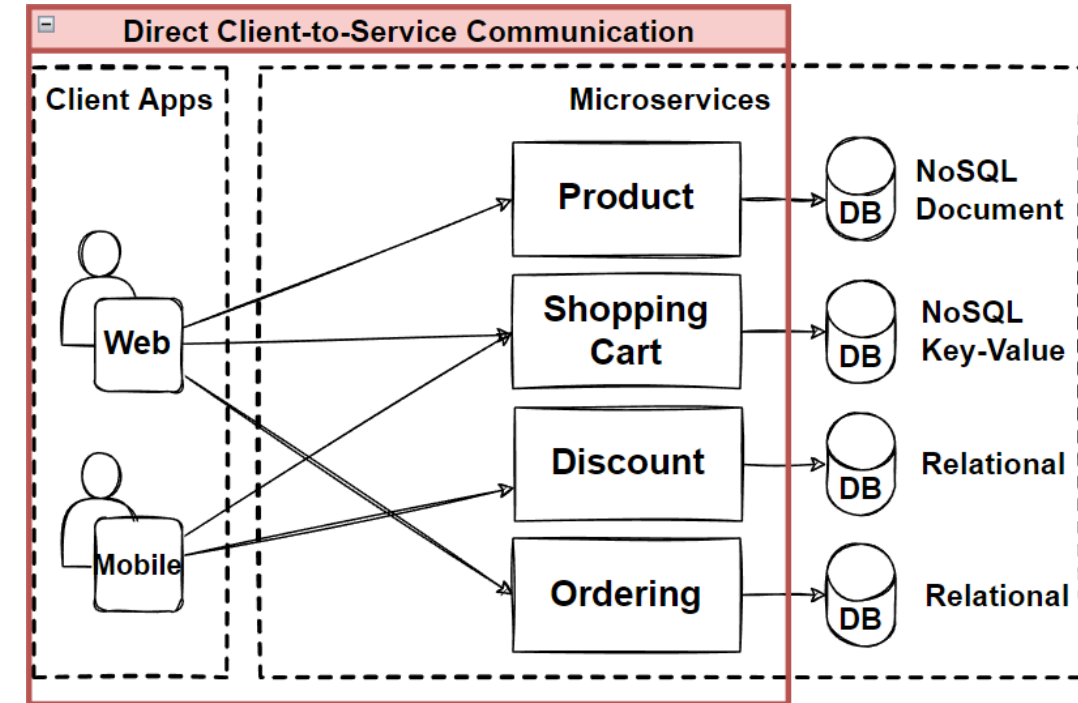
- Microservice Communications and Types
- Well-defined RESTful API Design for services
- **Microservices Communication Patterns (API Gateway, BFF, Publish/Subscribe..)**



Problems of Direct-to-Microservices Communication



- **Drawbacks** of the **direct client-to-microservices** communications, Comparison with API Gateway and direct client-to-microservice communications.
- In diagram, each microservices **has to open public endpoint** to the client applications but it has lots of drawbacks.
- If it is **small microservice-based application**, and if you don't have so much client applications, it could be good fit to use direct client-to-microservices communication.
- When you build **large and complex microservice-based** applications, if you have **dozens of microservices**, and several client applications that need to communicate multiple microservices.
- Direct-to-microservices communication can **creates major problems**.



Main Reasons of Why should use API Gateway

- **Client Can't Manage Communications**

The client applications can try to handle multiple calls to microservice endpoints, not manageable when it comes to communicates with different protocols like http, GraphQL, gRPC and WebSocket.

- **Complex Client Code**

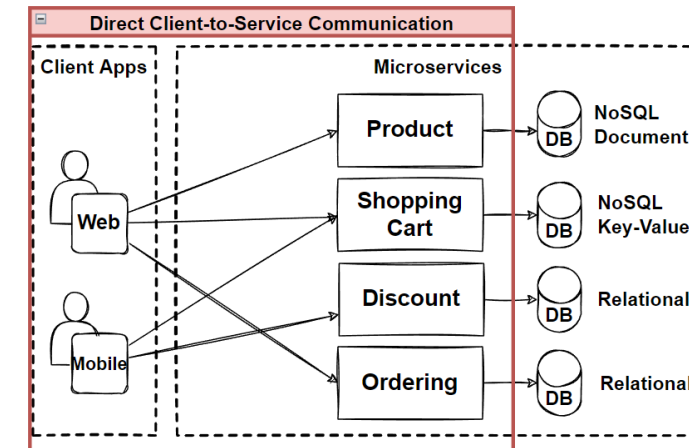
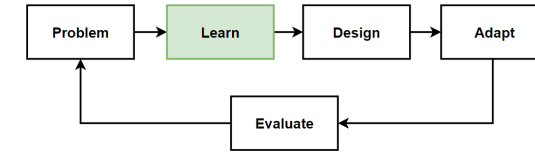
Cause to lots of requests to the backend services, create possible chatty communications. Increases latency and complexity on the UI side. Becomes complex client code.

- **Strong Coupling**

Creates coupling between the client and the backend. The client needs to know how the specific microservices are decomposed. Increase the coupling between client and services and also makes it harder to maintain the client.

- **Cross-cutting Concerns**

Authentication, Authorization, Rate Limiting, SSL certifications, Logging, Monitoring, Load Balancing and Circuit Breaker. Implementing all these cross-cutting concerns for every microservice is not a good solution.



Main Reason why should use API Gateway ? -2

- **Protocol Exchanges**

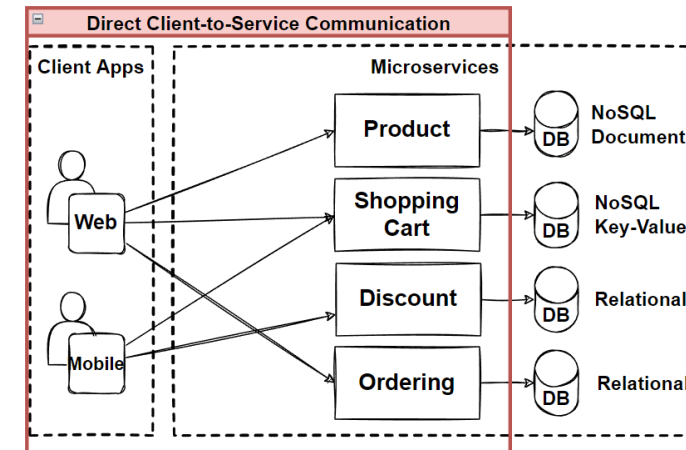
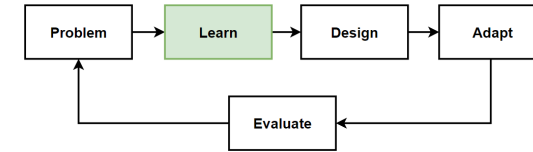
Communication required to use different protocols like exposing HTTP Rest API but continue to gRPC for internal communications or exposing WebSocket protocol. Requests must be translate to the other protocols afterwards.

- **Async Communication Requirements**

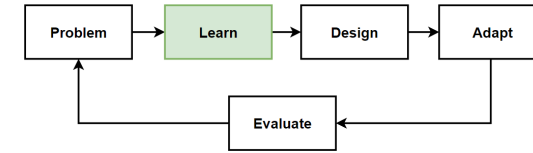
Client application can't use async communication due to not supported AMQP protocols. Microservices uses async communication in order to decouple communication as soon as possible. Hard to implement event-driven publish-subscribe model.

- **Payload Changes for Different Clients**

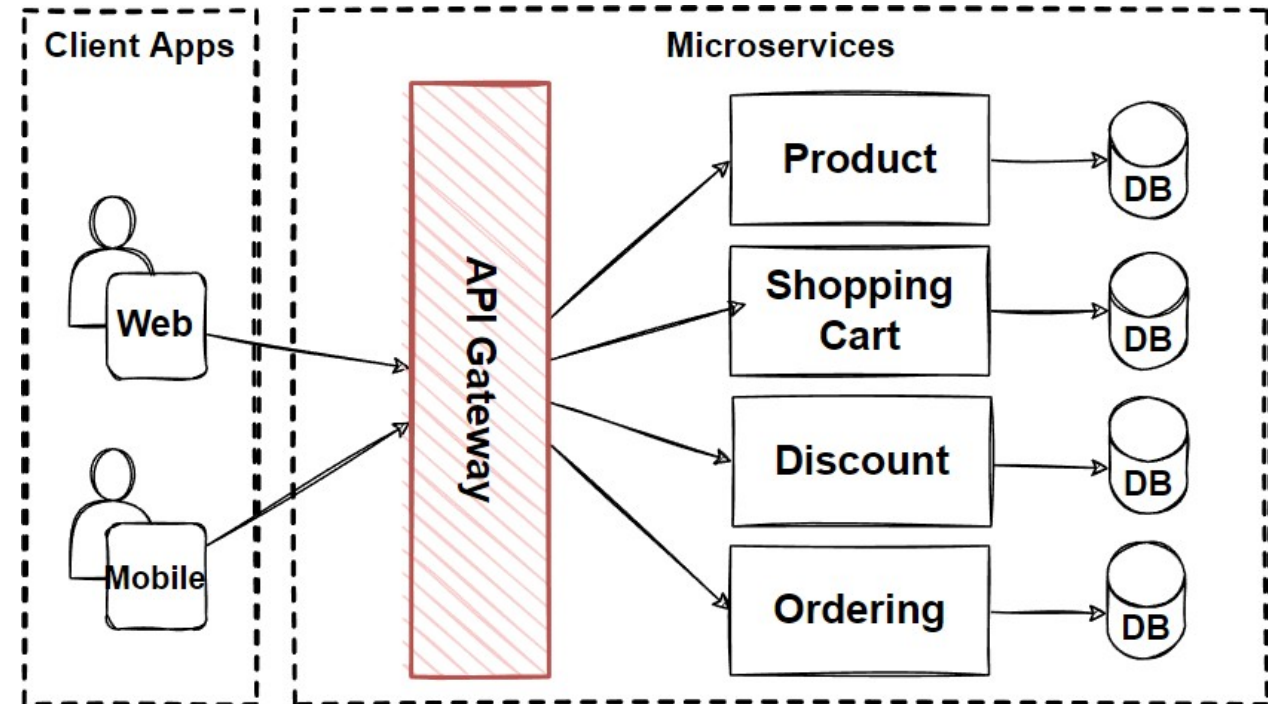
Web, Mobile and other client applications might be required different payloads when communicating with internal services. It requires to optimize data responses from services tailored with client applications.



The Solution - API Gateway Patterns

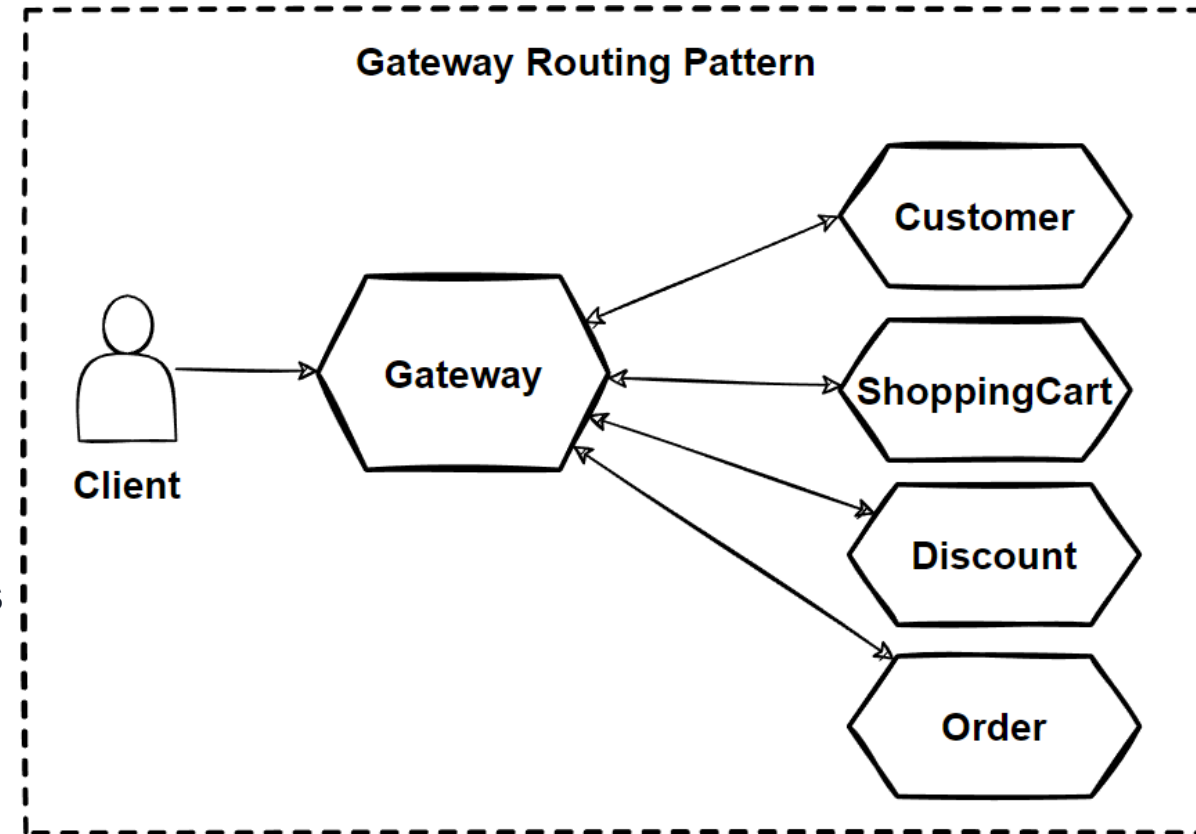
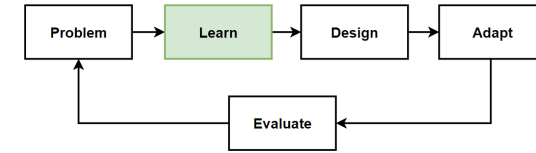


- Use **API Gateways** between client and internal microservices and it is **a single point of entry** to the client applications.
- **API Gateway** sits between **the client** and **multiple backend services** and manage **routing** to internal microservices.
- **API Gateways** can handle that **Cross-cutting concerns** like authentication, authorization, protocol translations, Rate Limiting, Logging, Monitoring, Load Balancing.
- API Gateway Patterns:
 - Gateway Routing pattern
 - Gateway Aggregation pattern
 - Gateway Offloading pattern



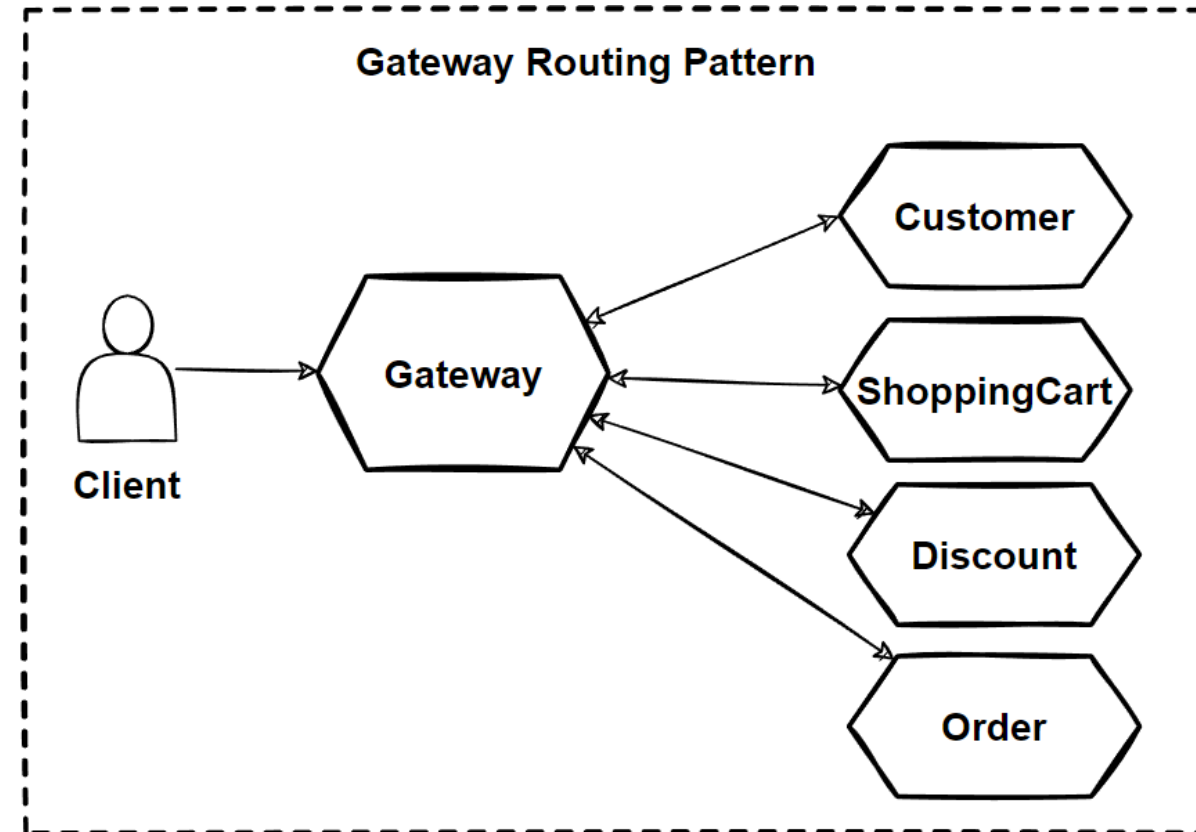
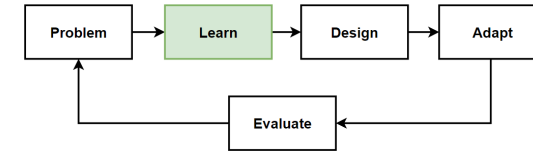
Gateway Routing pattern

- **Route requests to multiple microservices** with exposing a single endpoint.
- Useful when expose multiple services on a **single endpoint** and **route** them to **internal backend microservices** based on the request.
- The client needs to **consume several microservices**, **Gateway Routing pattern** offers to create a new endpoint that handle the request and route this request for each service.
- **E-commerce application** might provide services such as search Customers, Shopping cart, discount, and order history.
- **If one of microservices are changed**, the **Client doesn't know anything** and not need to change any code on client side, the only changes will be configuration routing changes.



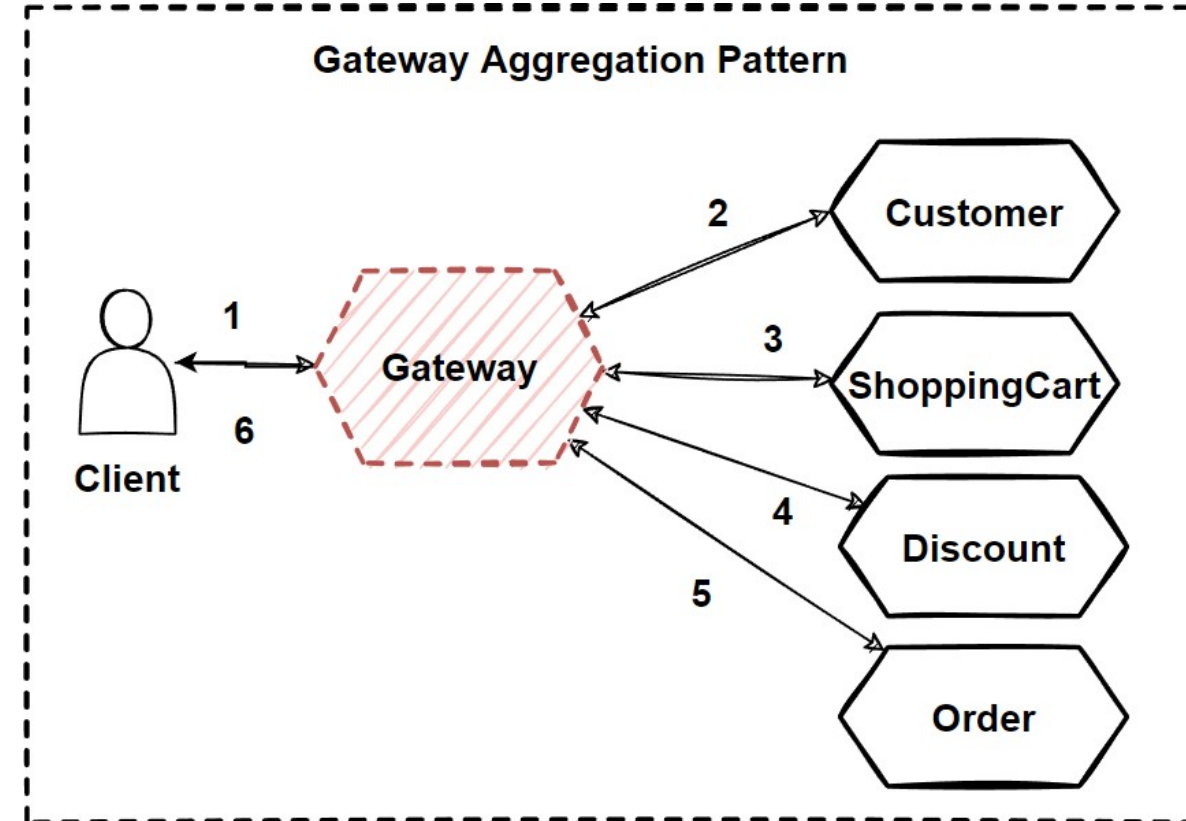
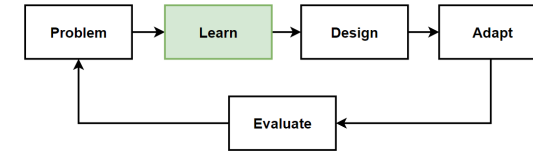
Gateway Routing pattern -2

- It **abstracts your backend microservices** from the client applications, with allowing to **keep client codes simple**, even backend services are changed on behind the API Gateway.
- **Deploy microservices APIs** with **blue/green** or **canary deployments** with multiple API versions of same microservices, you can **gradually shift your request** to new API with using the Gateway Routing Pattern.
- **Gateway Routing Pattern** gives you the **flexibility** to use different API versions routing to the requests.
- If new version of API has got exceptions, it can be quickly **reverted back** by making only a configuration change.
- Use **Application Layer 7 routing** to route the request to the internal services.

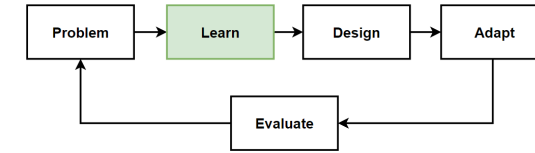
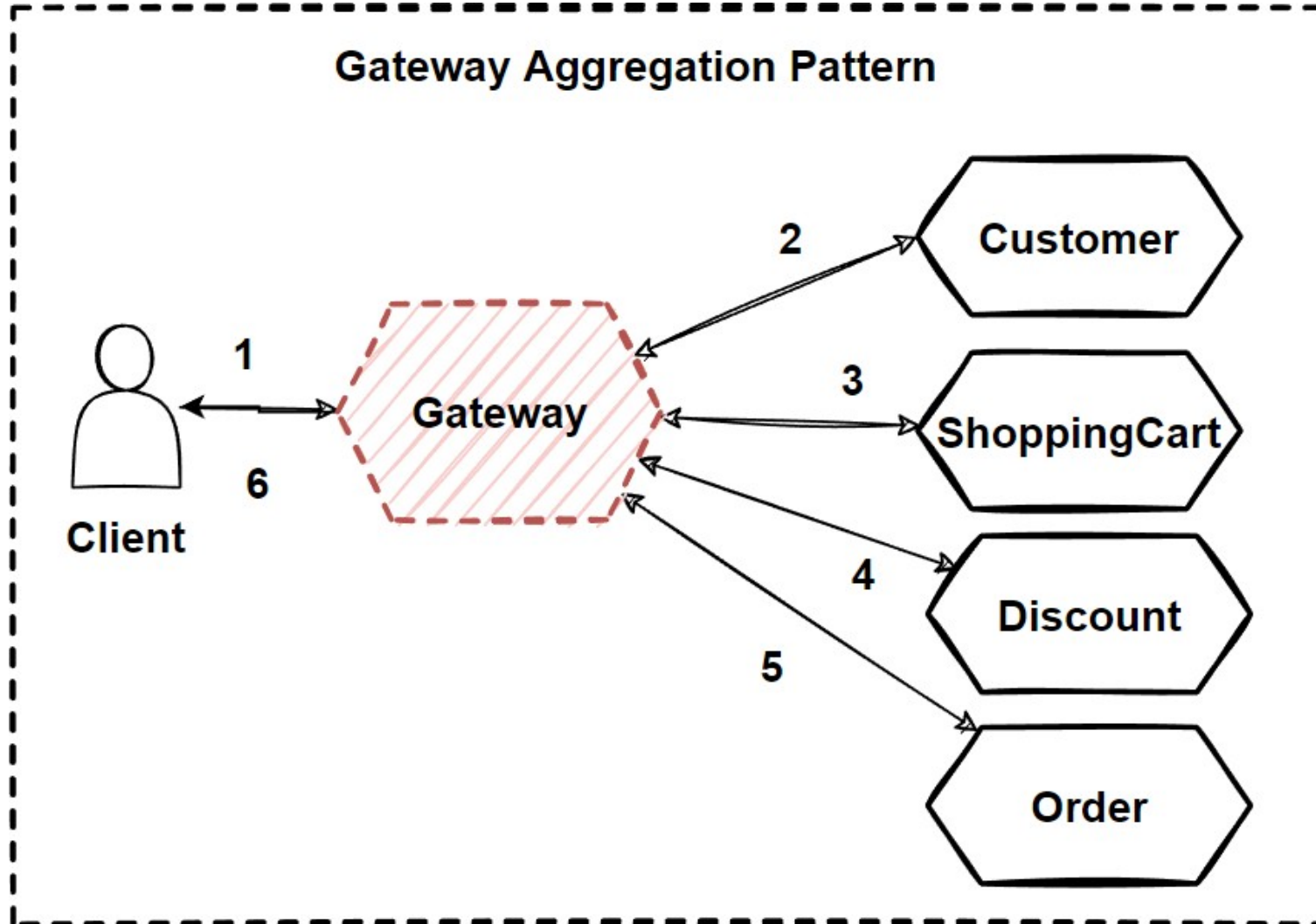


Gateway Aggregation Pattern

- API Gateway service that provide to **aggregate multiple individual requests** towards to internal microservices with **exposing a single request** to the client.
- Use if client application have to **invoke several different** backend microservices to perform its logic.
- Client applications need to **make multiple calls** to different backend microservices, this leads to **chattiness communications** and impact the performance.
- If **new service added** into use cases, then client need to send **additional request** that increase the network calls and latency.
- To **abstract complex** internal backend service communication from the client applications.
- **Dispatches requests** to the several backend services, **then aggregates** the results and sends them back to the requesting client.

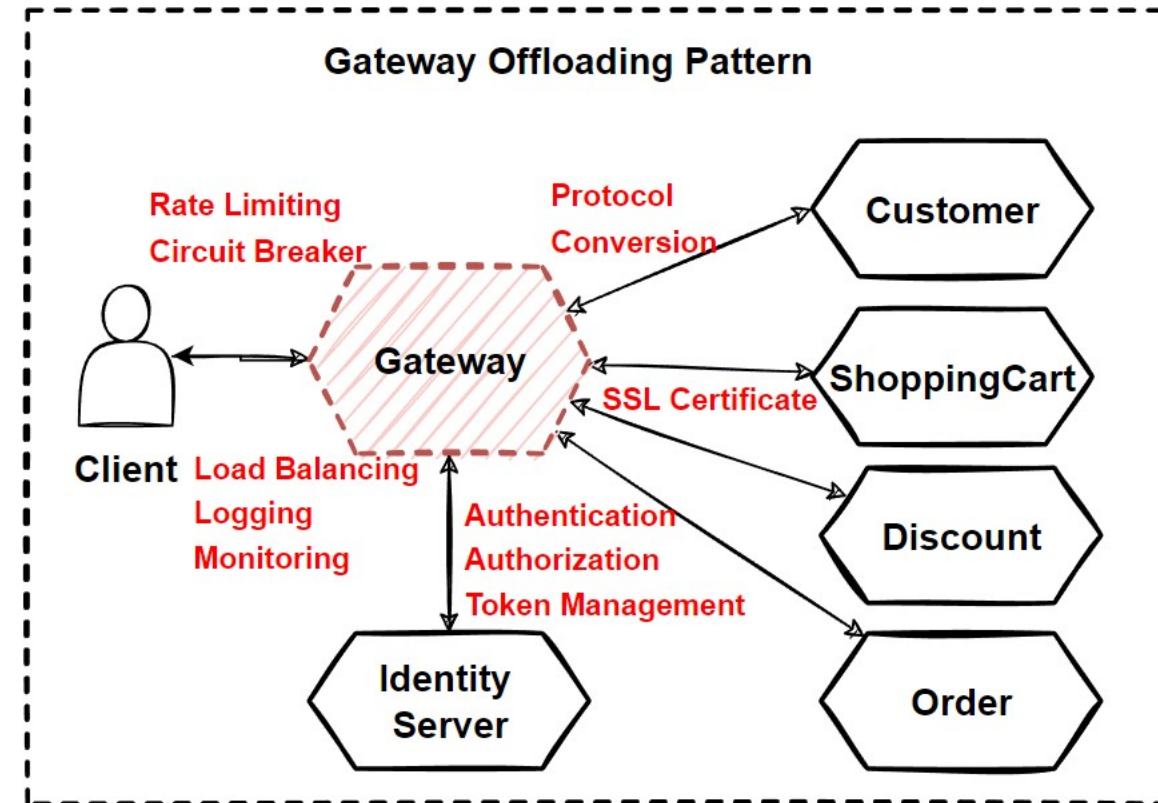
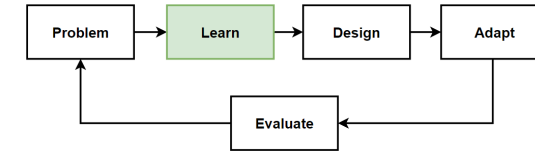


Gateway Aggregation Pattern-2

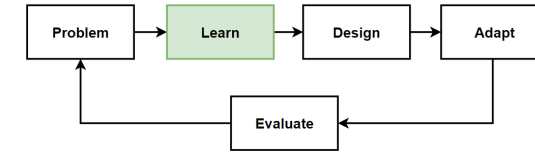
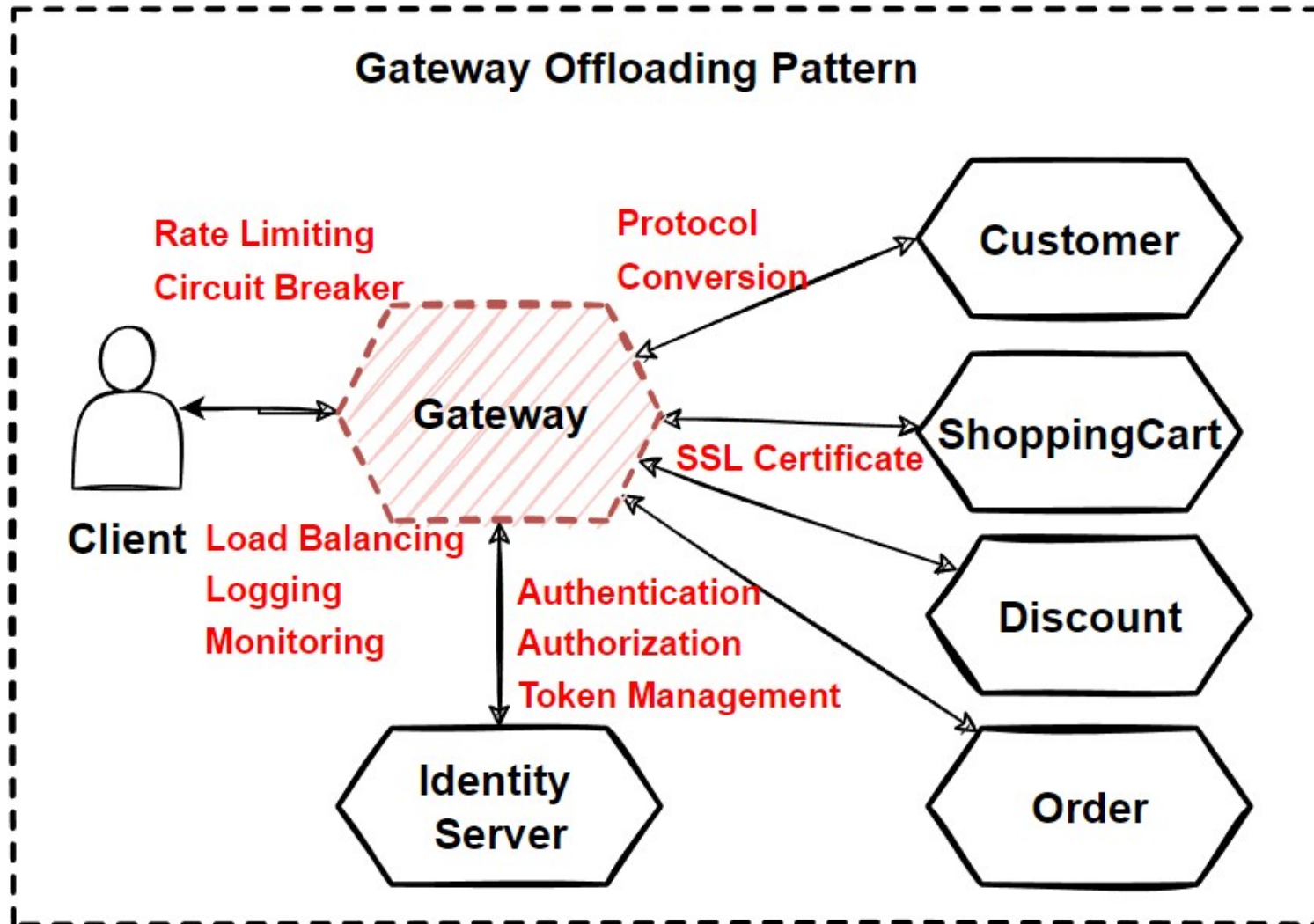


Gateway Offloading Pattern

- **Gateway Offloading Pattern** offers to combine commonly used shared functionalities into a gateway proxy services.
- **Shared functionalities** can application development by moving shared service functionality into centralized places.
- Authentication, Authorization, Rate Limiting, SSL certifications, Logging, Monitoring, Load Balancing.
- **Implementing cross-cutting concerns** for every microservice is **not a good solution**.
- Gateway Offloading Pattern offers to manage all those **Cross-cutting Concerns** into **API Gateways**.
- **Simplify the development of microservices** by removing the cross-cutting concerns on services to maintain supporting resources, allow to **focus on the application** functionality.

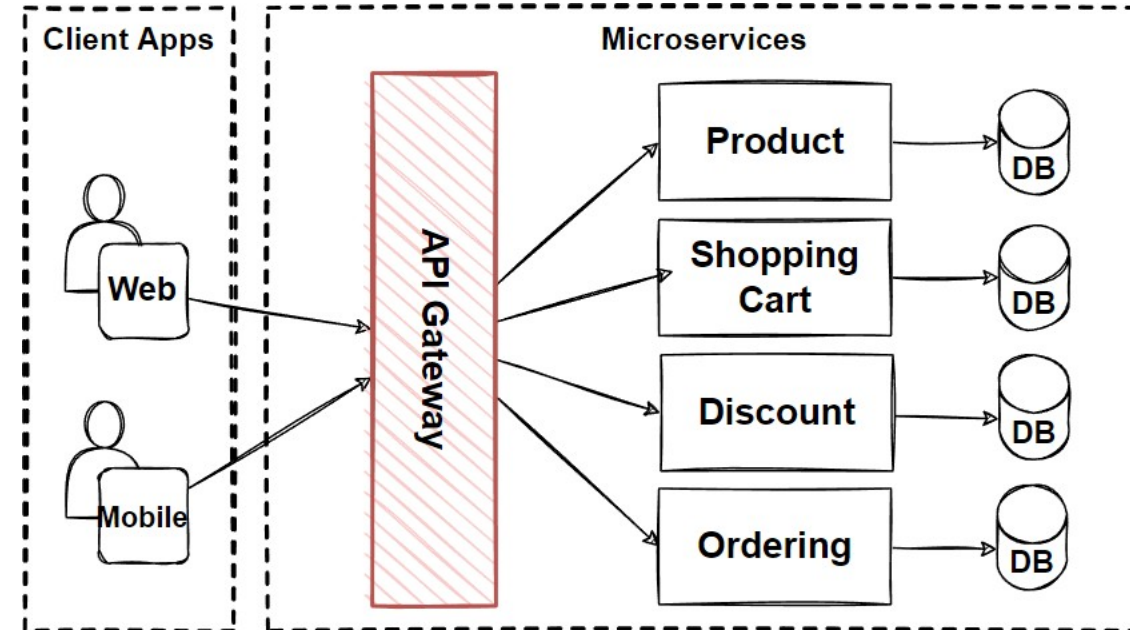
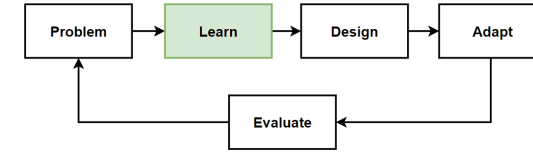


Gateway Offloading Pattern -2



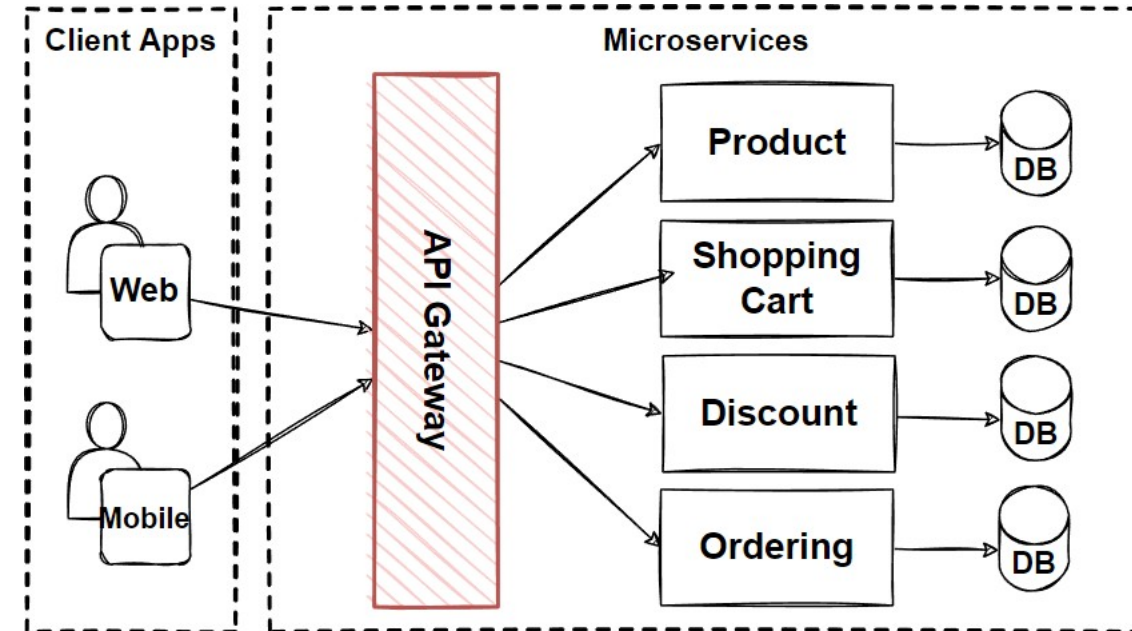
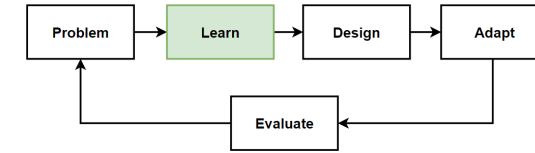
API Gateway Pattern

- **API Gateway** is a **single point of entry** to the client applications, sits between the **client** and **multiple backend**.
- API Gateways **manage routing** to internal microservices and able to **aggregate several microservice** request in 1 response and handle **cross-cutting concerns**.
- Recommended when design **complex large** microservices-based applications with **multiple client** applications.
- Similar to the **facade pattern** from object-oriented design, but it is a distributed system **reverse proxy or gateway routing** for using in synchronous communication.
- The pattern provides a **reverse proxy to redirect or route requests** to your internal microservices endpoints.
- API Gateway provides a **single endpoint** for the client applications, and it **maps the requests** to internal microservices.

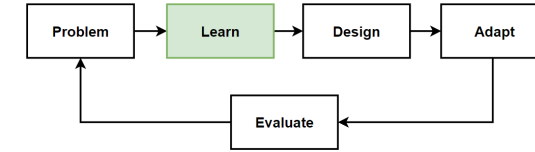


API Gateway Pattern - Summarized

- API gateway **locate between** the **client** apps and the **internal microservices**.
- Working as a **reverse proxy** and **routing requests** from clients to backend services and provide **cross-cutting concerns** like authentication, SSL termination, and cache.
- Several client applications connect to single API Gateway possible to **single-point-of-failure risk**.
- If these client applications increase, or adding more logic to **business complexity** in API Gateway, it would be **anti-pattern**.
- Best practices is **splitting** the **API Gateway** in multiple services or multiple smaller API Gateways: **BFF-Backend-for-Frontend Pattern**.
- Should **segreated based** on **business boundaries** of the client applications.



Main Features of API Gateway Pattern



- **Reverse Proxy and Gateway Routing**

Reverse proxy to redirect requests to the endpoints of the internal microservices. Using Layer 7 routing for HTTP requests for redirections. Decouple client applications from the internal microservices. Separating responsibilities on network layer and abstracting internal operations.

- **Requests Aggregation and Gateway Aggregation**

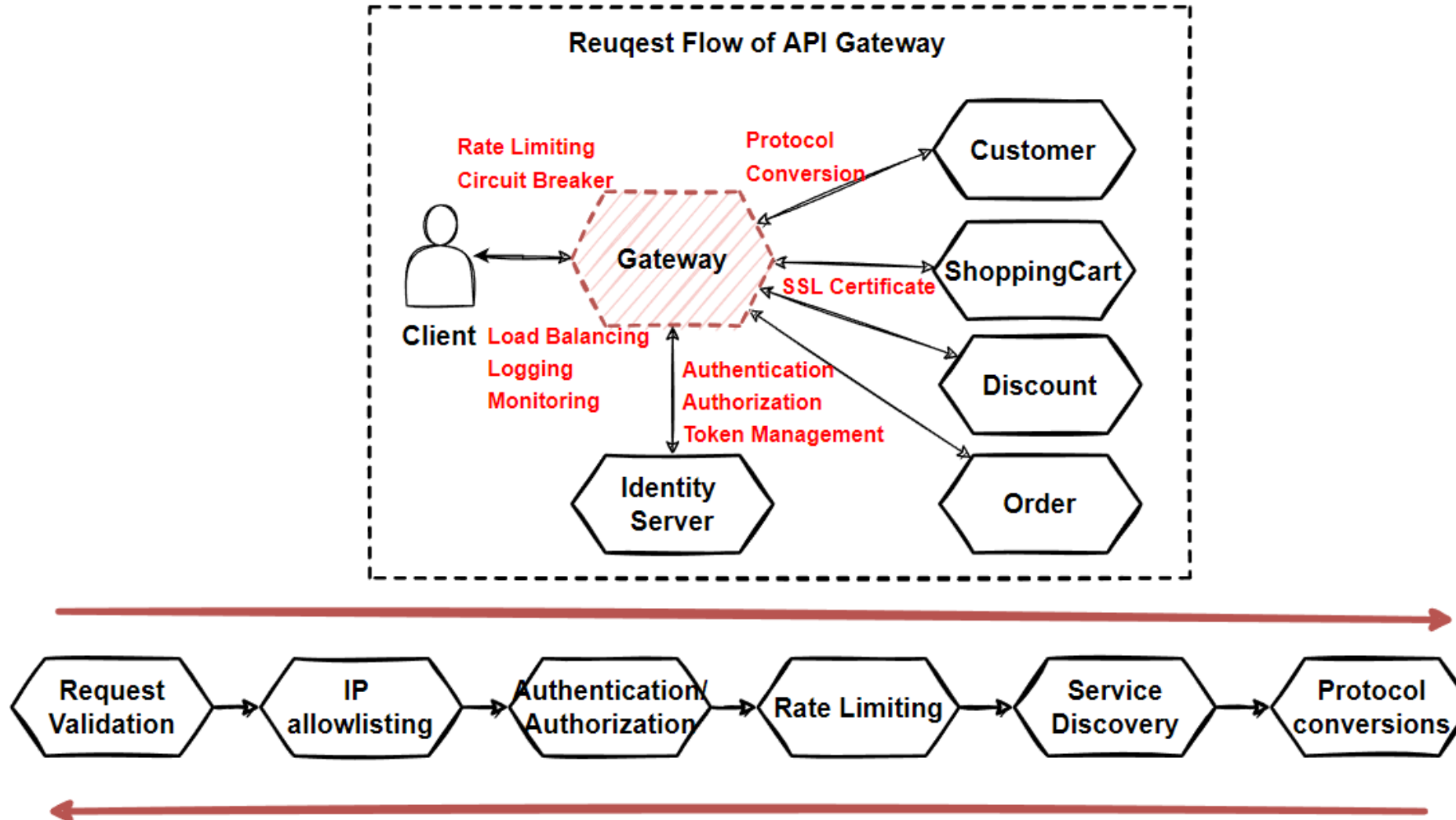
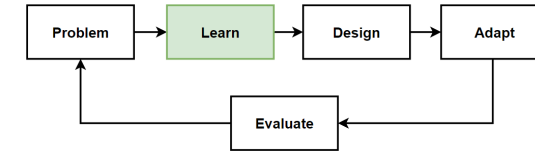
Aggregate multiple internal microservices into a single client request. Client application sends a single request to the API Gateway and it dispatches several requests to the internal microservices and then aggregates the results and sends back to the client application in 1 single response. Reduce chattiness communication.

- **Cross-cutting Concerns and Gateway Offloading**

Best practice to implement cross-cutting functionality on the API Gateways. Cross-cutting functionalities can be; Authentication and authorization, Service discovery, Response caching, Retry policies, Circuit Breaker, Rate limiting and throttling, Load balancing, Logging, tracing, IP allowlisting.

Routing	Authentication
Request Aggregation	Authorization
Service Discovery with Consul & Eureka	Throttling
Load Balancing	Logging, Tracing
Correlation Pass-Through	Headers/Query String Transformation
Quality of Service	Custom Middleware

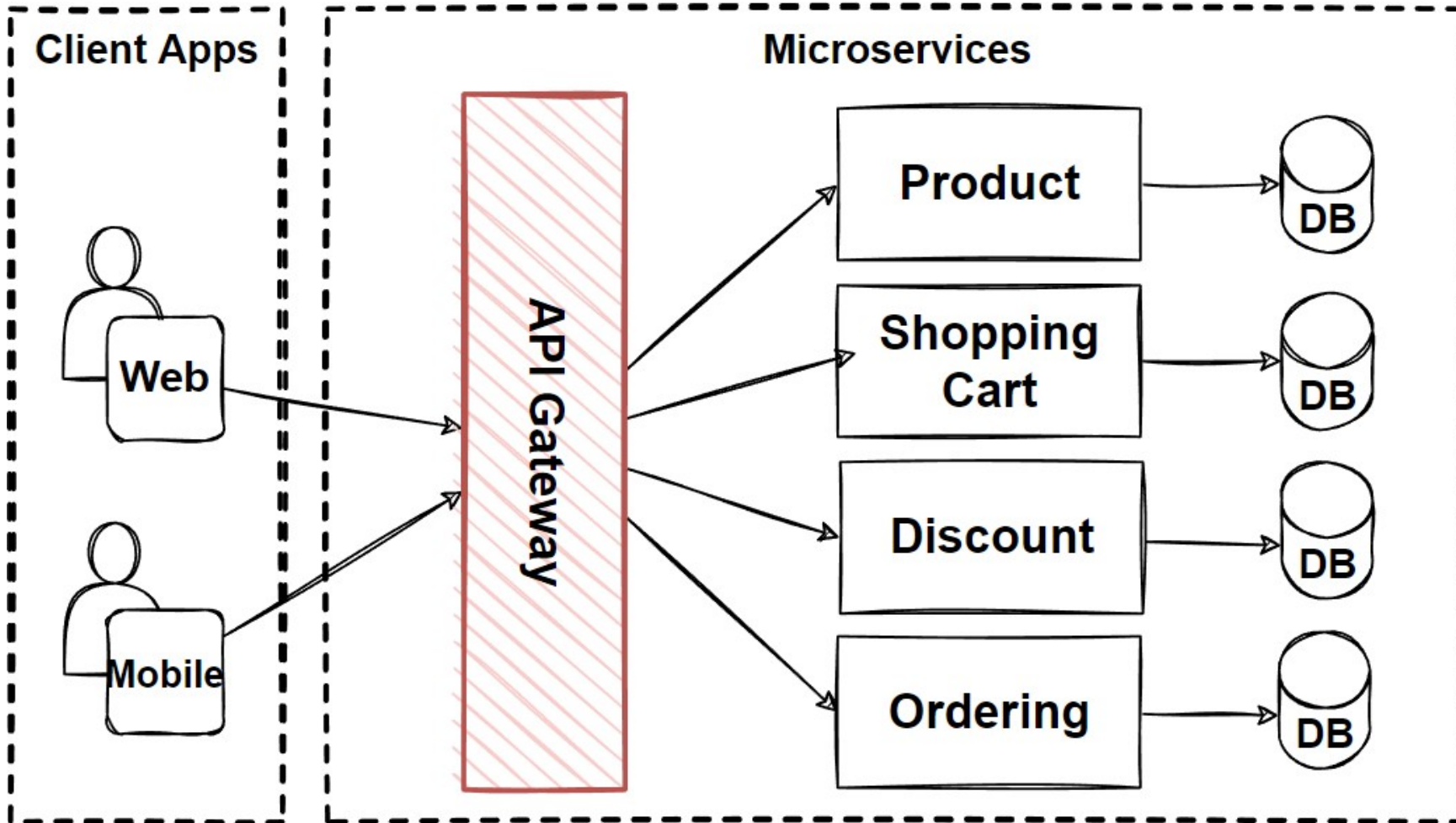
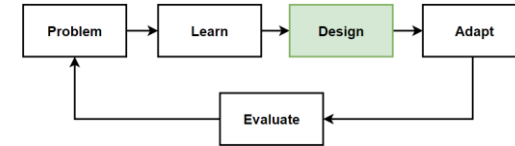
Request Flow of API Gateway Pattern



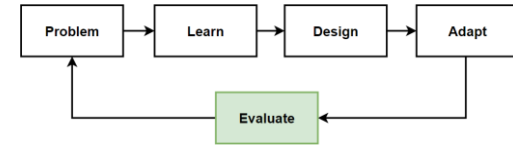
Before Design – What we have in our design toolbox ?

Architectures	Patterns&Principles	Microservices Communications	Non-FR	FR
• Microservices Architecture	• The Database-per-Service Pattern • Polygot Persistence • Decompose services by scalability • The Scale Cube • Microservices Decomposition Pattern • Microservices Communications Patterns	• HTTP Based RESTful API • GraphQL API • gRPC API • WebSocket API • Gateway Routing Pattern • Gateway Aggregation Pattern • Gateway Offloading Pattern • API Gateway Pattern	• High Scalability • High Availability • Millions of Concurrent User • Independent Deployable • Technology agnostic • Data isolation • Resilience and Fault isolation	• List products • Filter products as per brand and categories • Put products into the shopping cart • Apply coupon for discounts • Checkout the shopping cart and create an order • List my old orders and order items history

Design: Microservices Architecture with API Gateway



Evaluate: Microservice Architecture with API Gateway

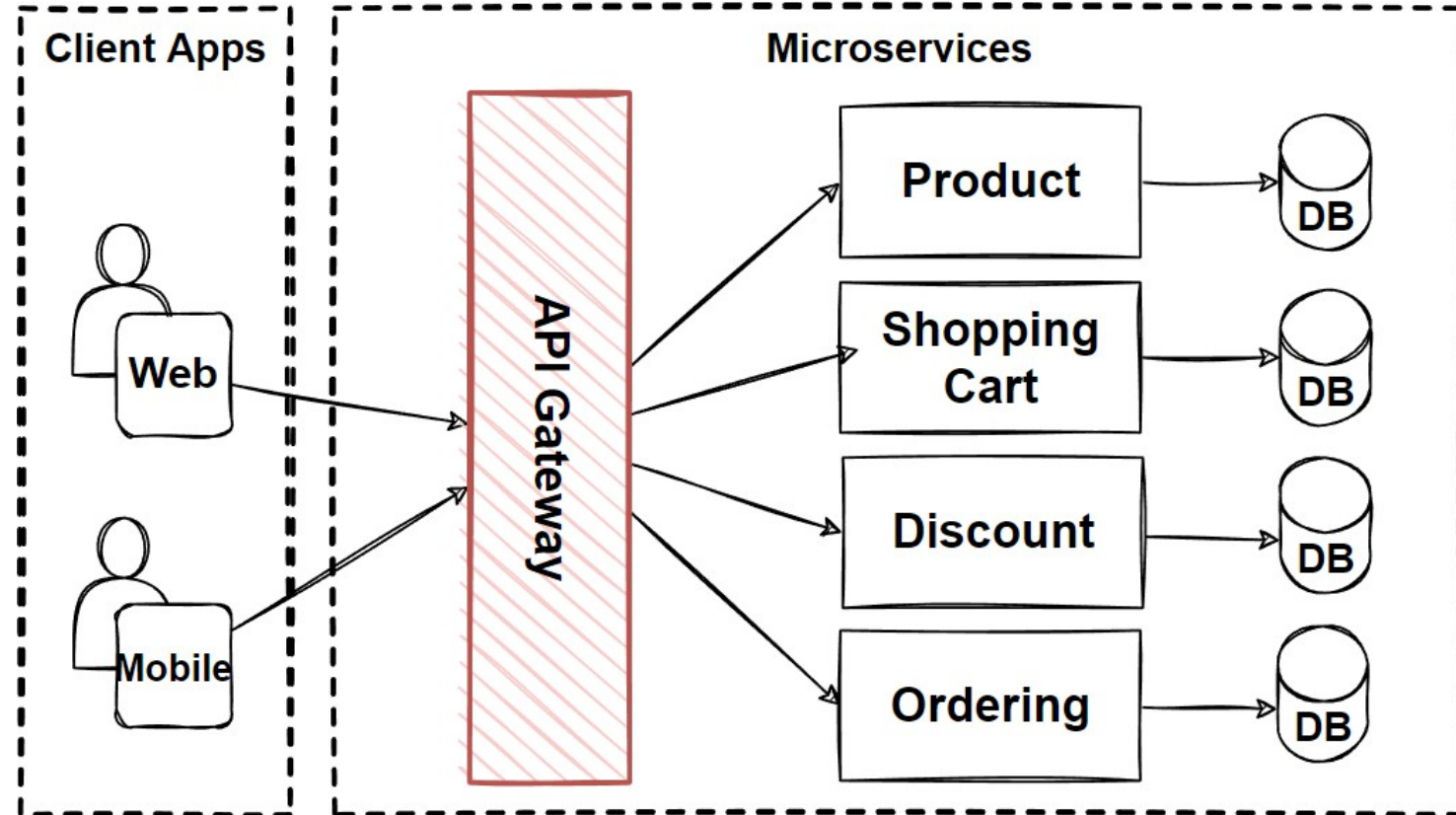


Benefits

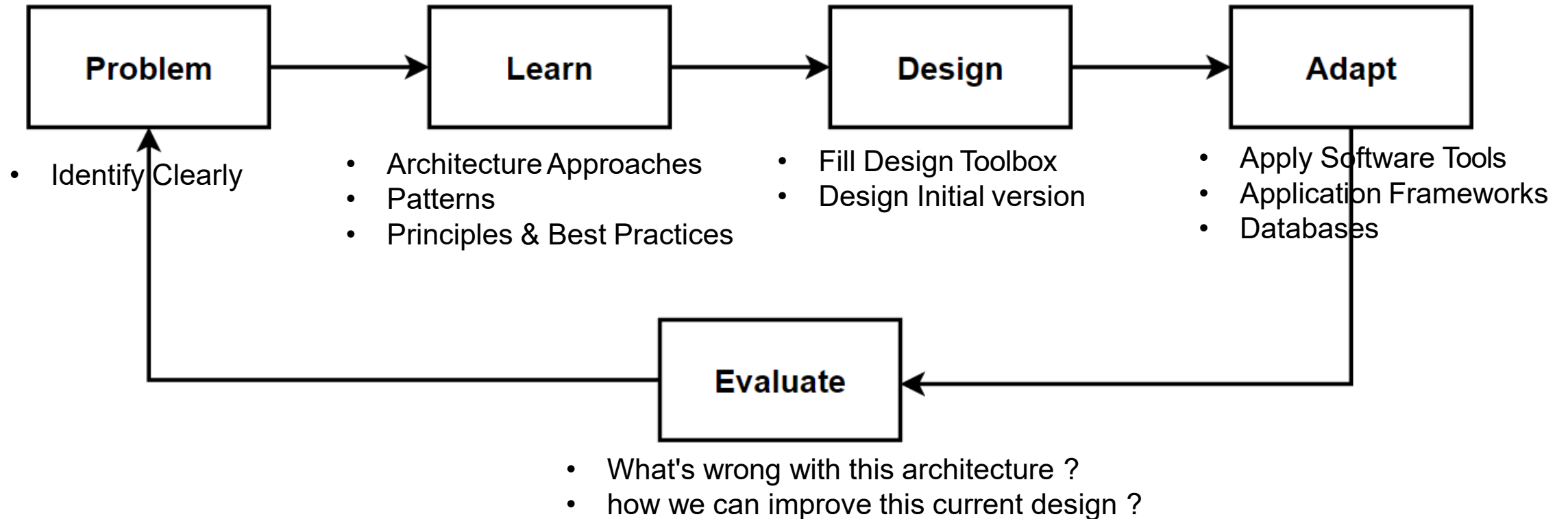
- Reverse proxy, Routing, Aggregation
- Abstraction Backend services
- Reduces chattiness network traffic
- Implements Cross-cutting concerns

Drawbacks

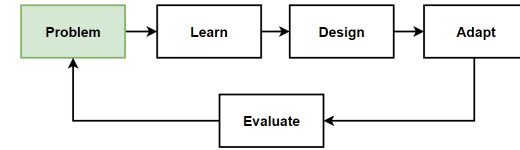
- Single API Gateway = single-point-of-failure risk
- Business complexity in API Gateway
- If not scale out, API Gateway can become a bottleneck on network traffic



Way of Learning – The Course Flow



Problem: Client Apps has Different UI Requirements

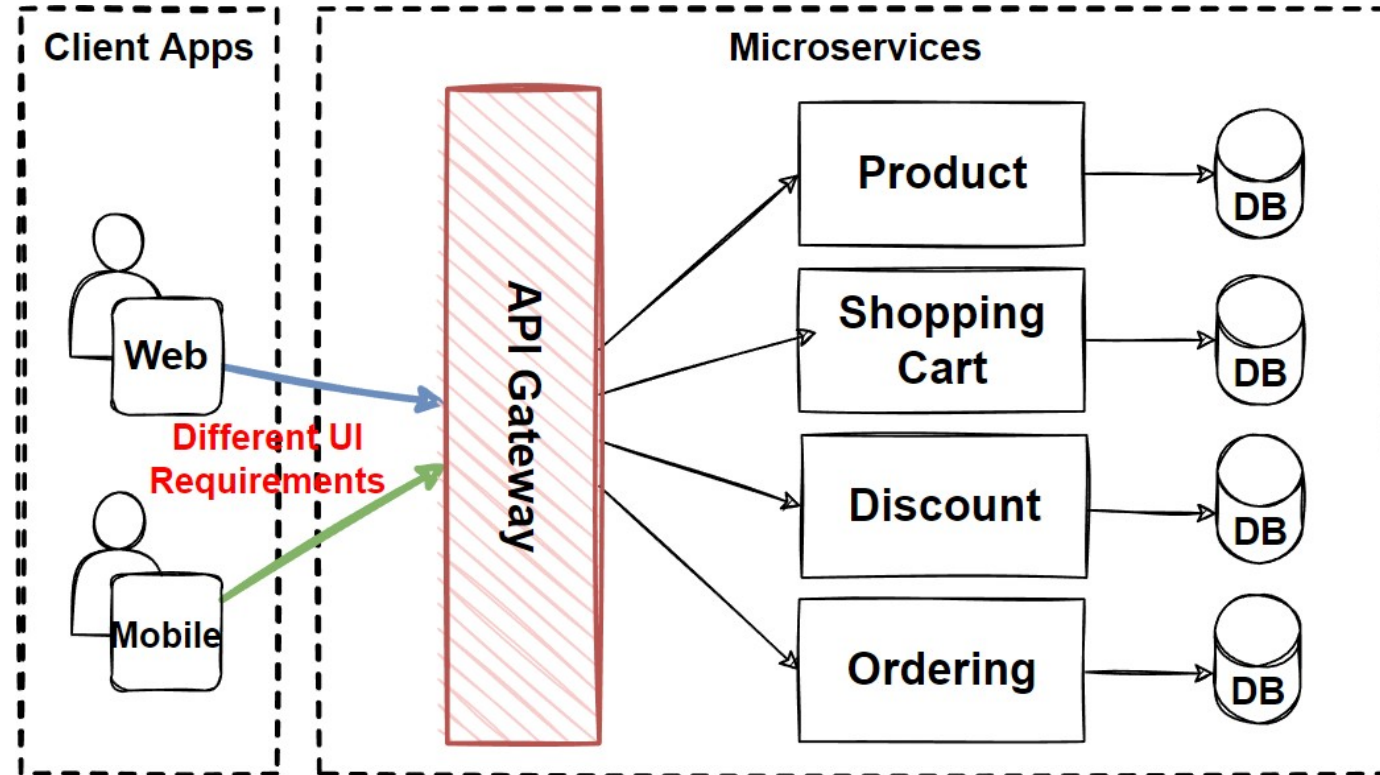


Problems

- Frontend client applications has different UI and Data consumption requirements
- Different data representation requirements need extra effort to adapt data for specific Client UI
- Clients code has custom logic to re-format these data.
- Client codes complex and hard to maintain for future developments. Also it increase browser's usage of high resources.

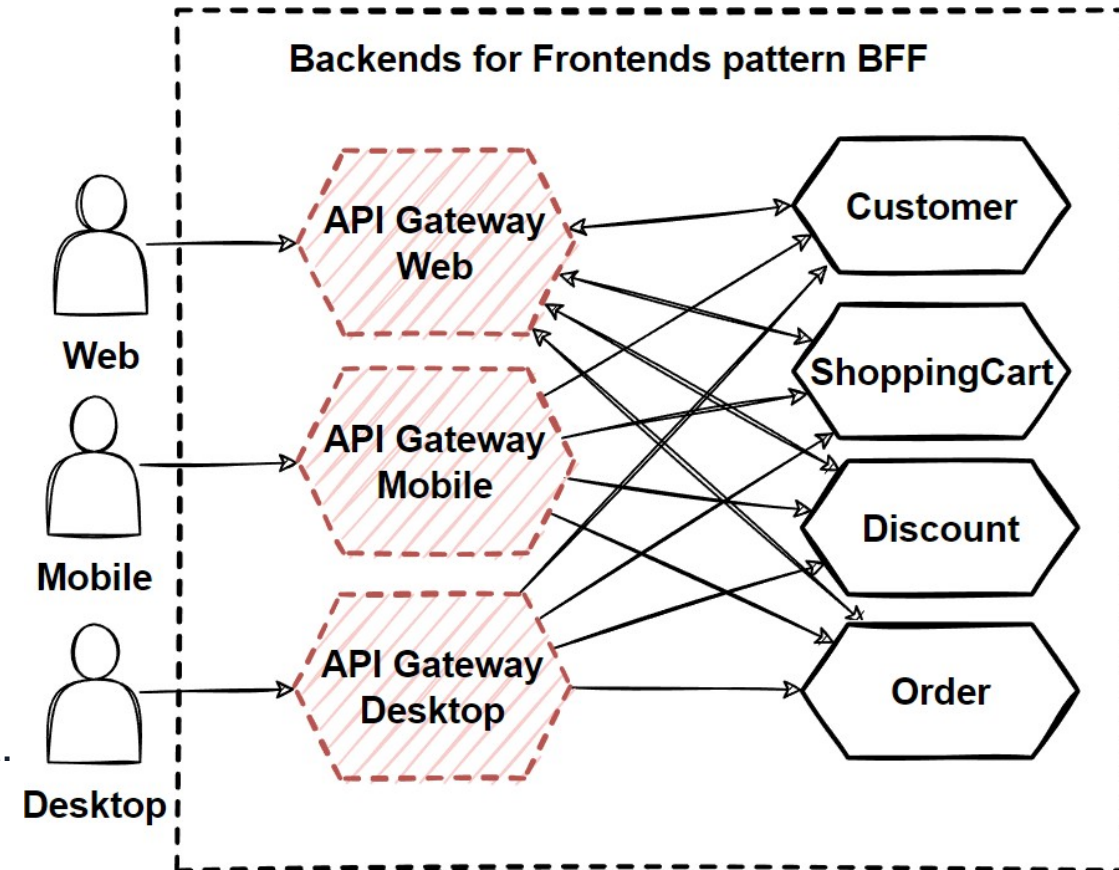
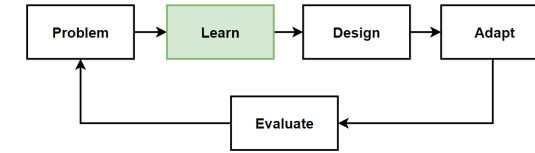
Solutions

- Client focused Interfaces
- **BFF-Backend for Frontend Pattern**

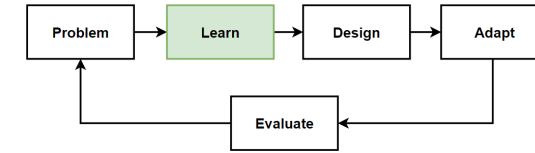
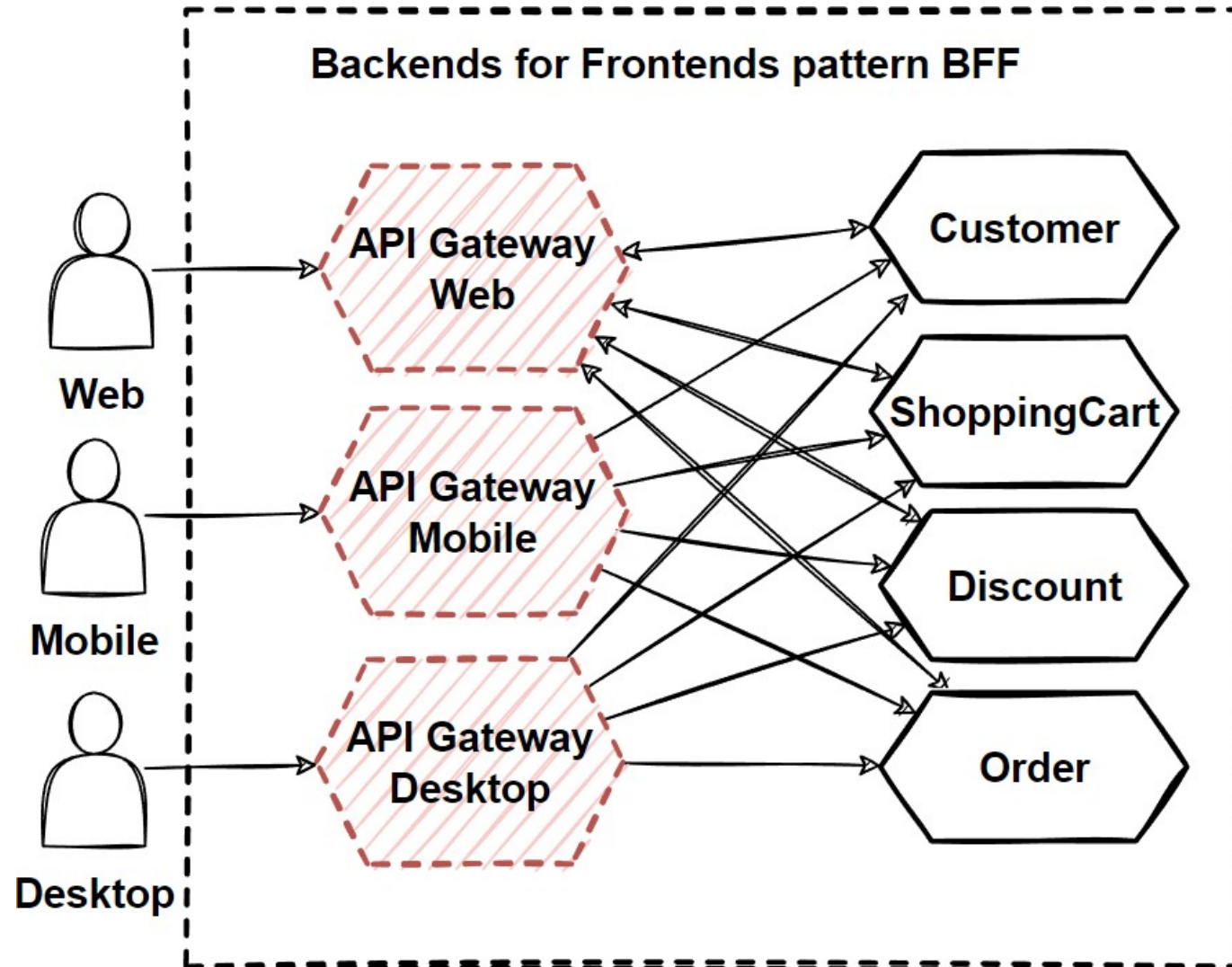


Backends for Frontends Pattern - BFF

- **Backends for Frontends** pattern basically **separate API Gateways** as per the **specific frontend** applications.
- Single API Gateway makes a **single-point-of failure**.
- BFF offers to create several API Gateways and **grouping** the **client applications** according to their boundaries.
- A **single** and **complex** API Gateway can be **risky** and becoming a **bottleneck** into your architecture.
- Larger systems often expose multiple API Gateways by **grouping client type** like mobile, web and desktop functionality.
- **Create several API Gateways** as per user interfaces to provide to best match the needs of the frontend environment.
- **BFF Pattern** is to provide client applications has a **focused interfaces** that connects with the internal microservices.



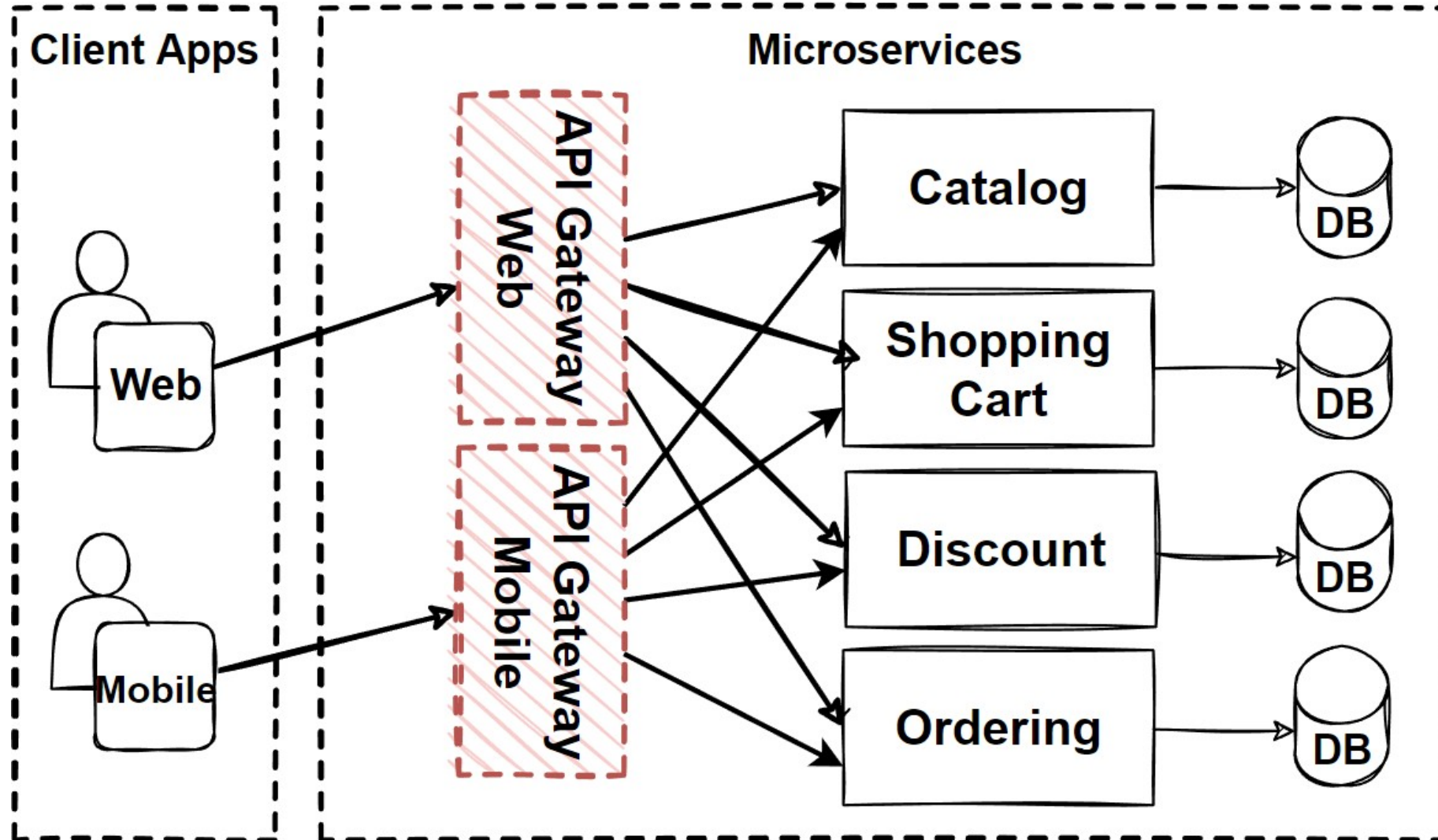
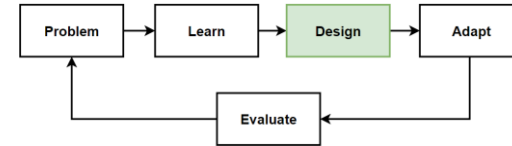
Backends for Frontends Pattern - BFF



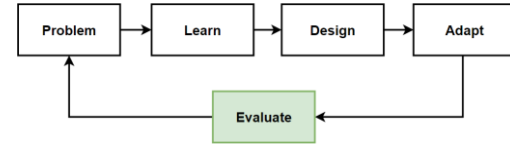
Before Design – What we have in our design toolbox ?

Architectures	Patterns&Principles	Microservices Communications	Non-FR	FR
• Microservices Architecture	• The Database-per-Service Pattern • Polygot Persistence • Decompose services by scalability • The Scale Cube • Microservices Decomposition Pattern • Microservices Communications Patterns	• HTTP Based RESTfulAPI • GraphQL API • gRPC API • WebSocket API • Gateway Routing Pattern • Gateway Aggregation Pattern • Gateway Offloading Pattern • API Gateway Pattern • Backends for Frontends Pattern-BFF	• High Scalability • High Availability • Millions of Concurrent User • Independent Deployable • Technology agnostic • Data isolation • Resilience and Fault isolation	• List products • Filter products as per brand and categories • Put products into the shopping cart • Apply coupon for discounts • Checkout the shopping cart and create an order • List my old orders and order items history

Design: Microservices Architecture with BFF



Evaluate: Microservice Architecture with BFF

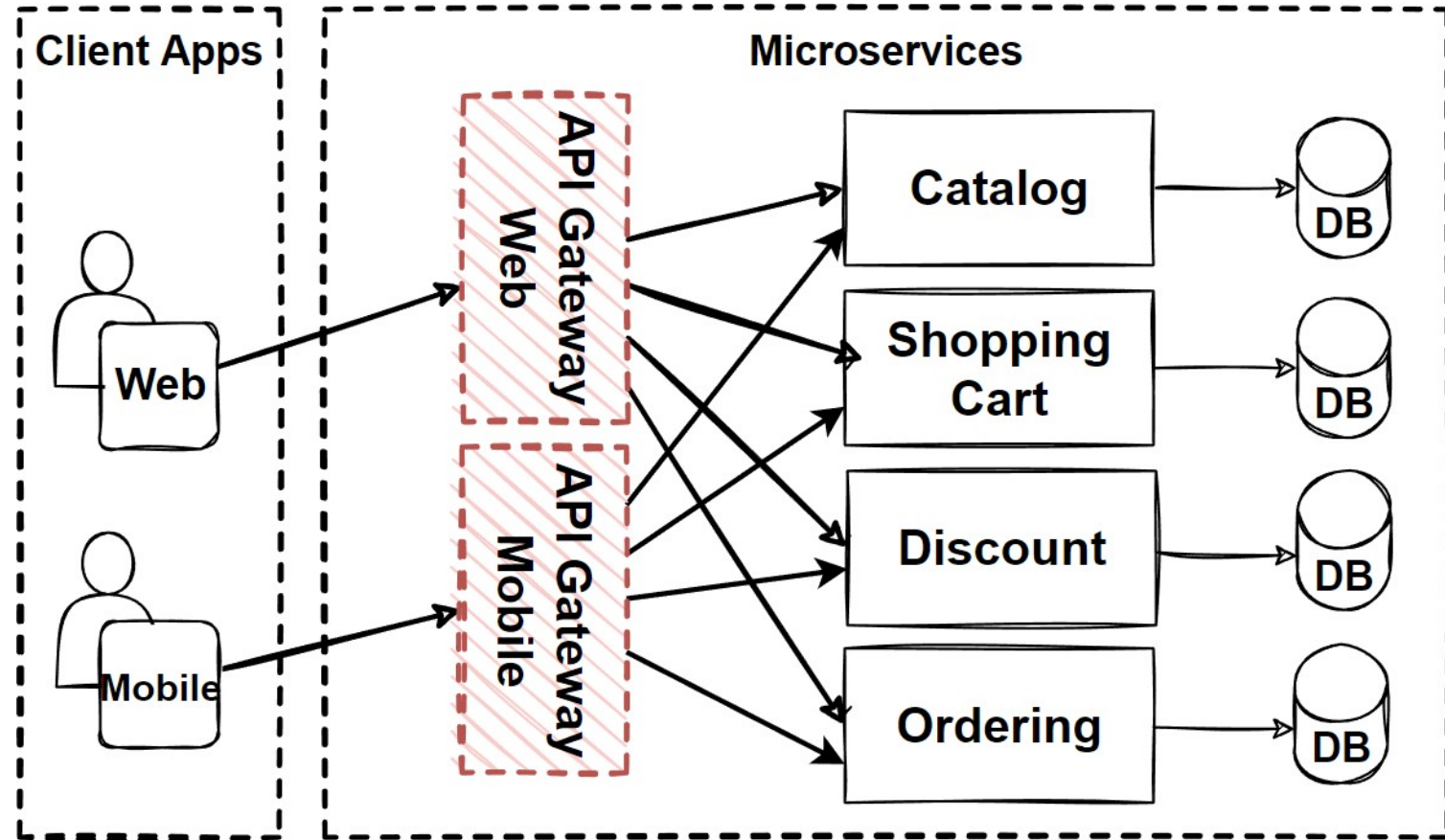


Benefits

- Client focused Interfaces
- Minimal logic on the frontend code
- Streamline data representation
- Well-focused interface for the frontend client applications

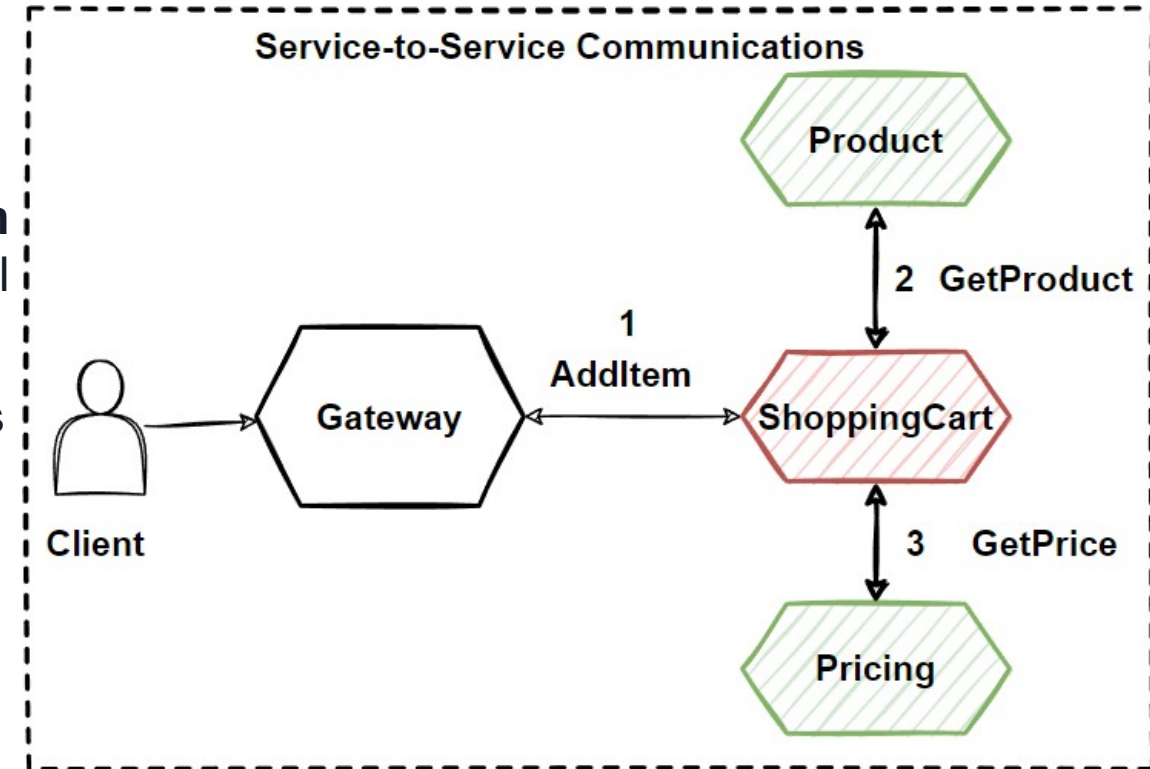
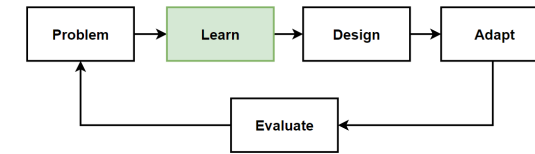
Drawbacks

- Increased Latency
- Good for large-scaled microservices application which has several client applications



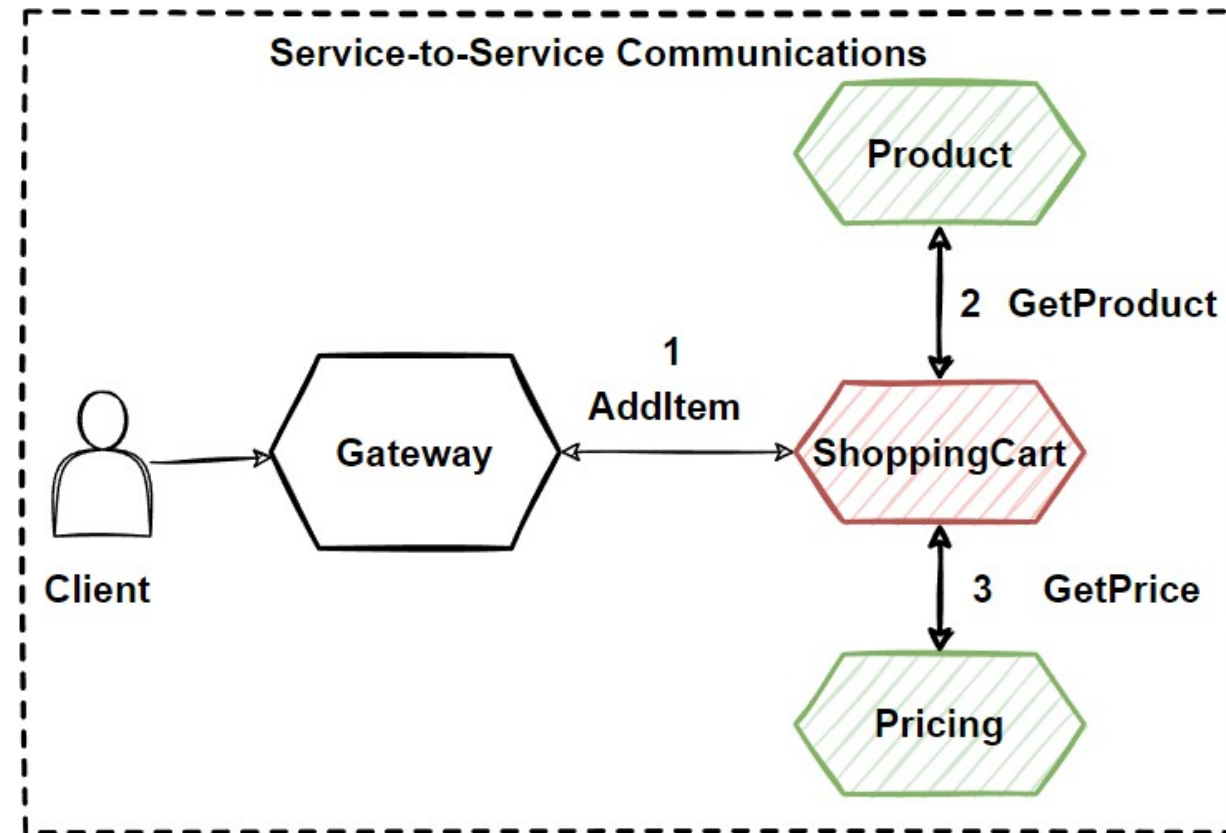
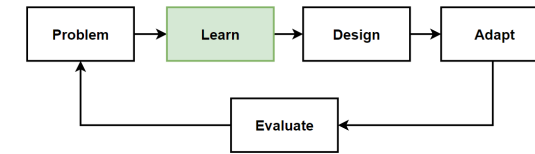
Service-to-Service Communications between Backend Internal Microservices

- Created **API Gateways** and separate this API Gateways following the **BFF Pattern**.
- **Sync request** comes from the clients and goes to internal microservices **over the API Gateways**.
- What if the **client requests** are **required to visit more than one** internal **microservices** ? How we can manage internal microservice communications ?
- Best practice is **reducing inter-service communication** as much as possible.
- In some cases, we can't reduce these internal communications due to operation **need to visit several internal services**.
- **Client send query request** to internal microservices to accumulate some data in the screen page of frontend.

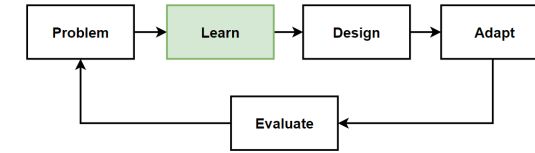


Service-to-Service Communications between Backend Internal Microservices-2

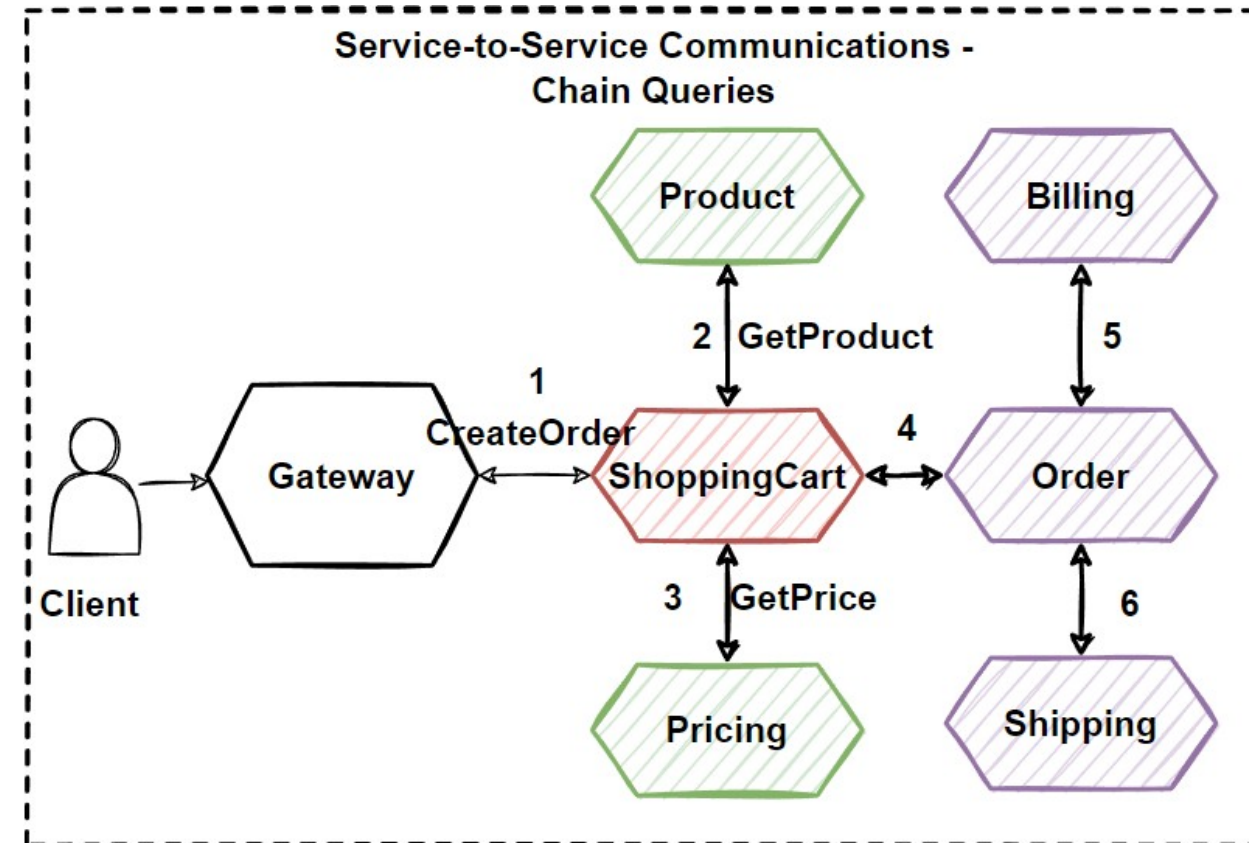
- User **addItem** into shopping cart, these operation need to get data from Product and Price microservices.
- **Synchronous communication** over the API Gateways which is the Request/Response Messaging.
- The client **send http request** to the microservices for querying or adding item into Shopping Cart:
 - 1- The client send request to API Gateway
 - 2- API Gateway dispatch request to ShoppingCart
 - 3- SC need to extend information by sending sync request to Product and Pricing microservices
- **Internal calls** makes **coupling** each microservices.
- If one of the **microservices** is **down**, it can't return data to the client so **its not any fault-tolerance**.



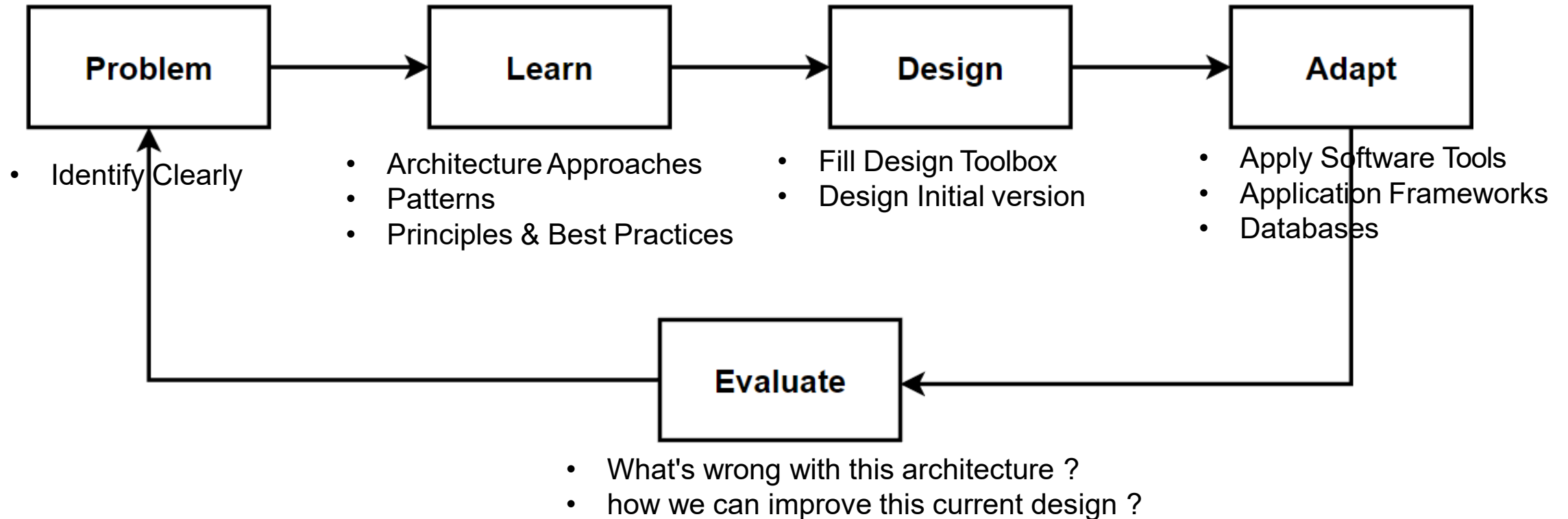
Service-to-Service Communications Chain Queries



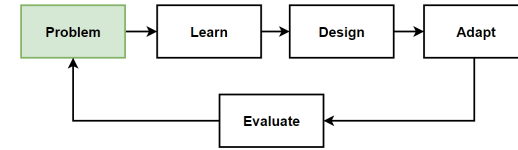
- If **service calls** are much then a few HTTP calls to multiple microservices, than it goes to **un-manageable** situation.
- **Place Order Use Case:** client checkout shopping cart, start order fulfillment processes.
- Request/Response Sync Messaging end up **with 6 sync chain HTTP Request.**
- **Increase latency** and **negatively impact** the performance, scalability, and availability.
- What if the **step 5 or 6 is failed ?**
- We can **apply 2 way** to solve this issues:
 - 1- Change microservices communications to async way
 - 2- Use Service Aggregator Pattern to aggregate somequery operations in 1 API Gateway.



Way of Learning – The Course Flow



Problem: Service-to-Service Communications Chain Queries

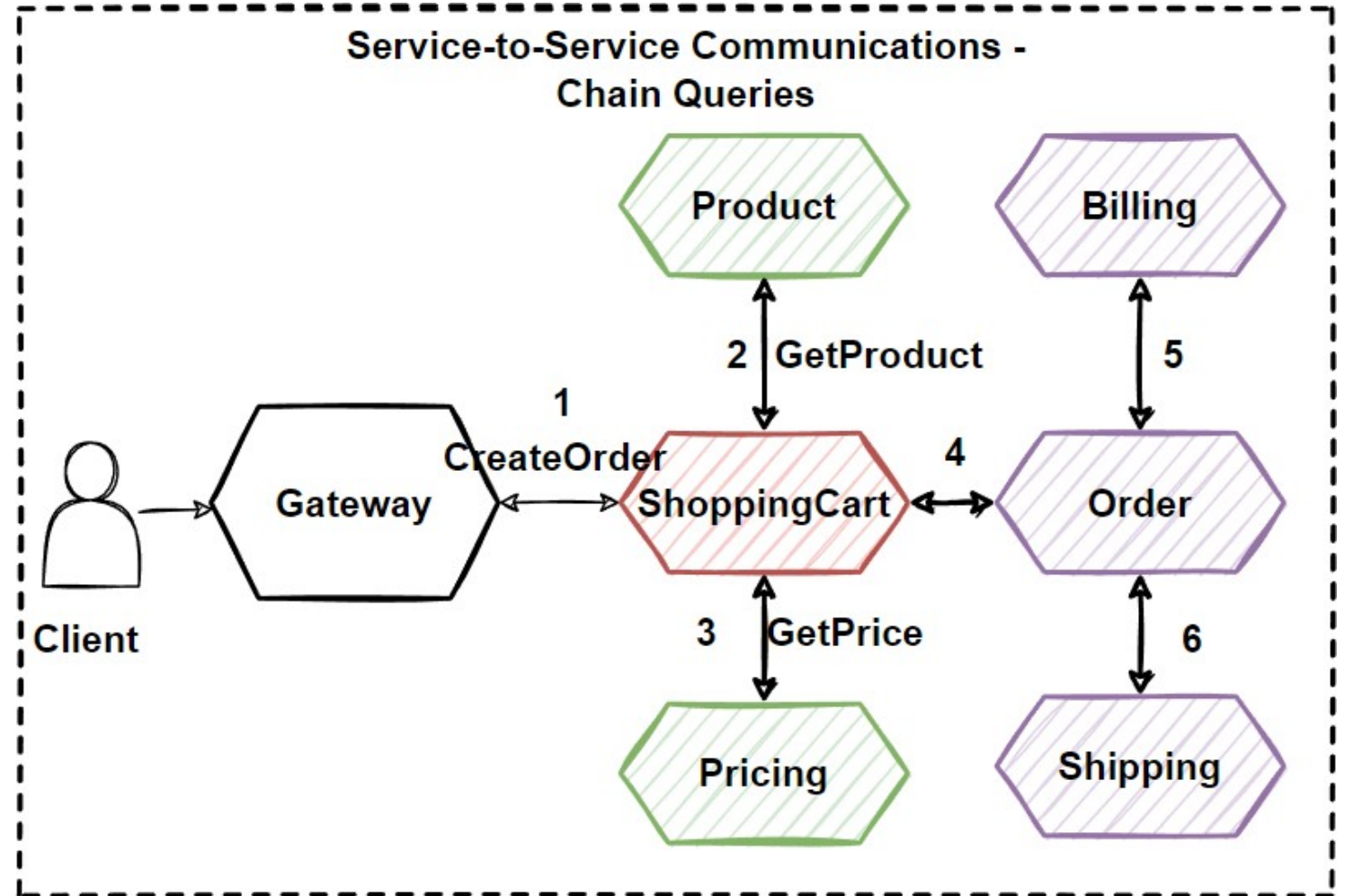


Problems

- HTTP calls to multiple microservices
- Chain Queries
- Visit more than a few microservices
- Increased latency

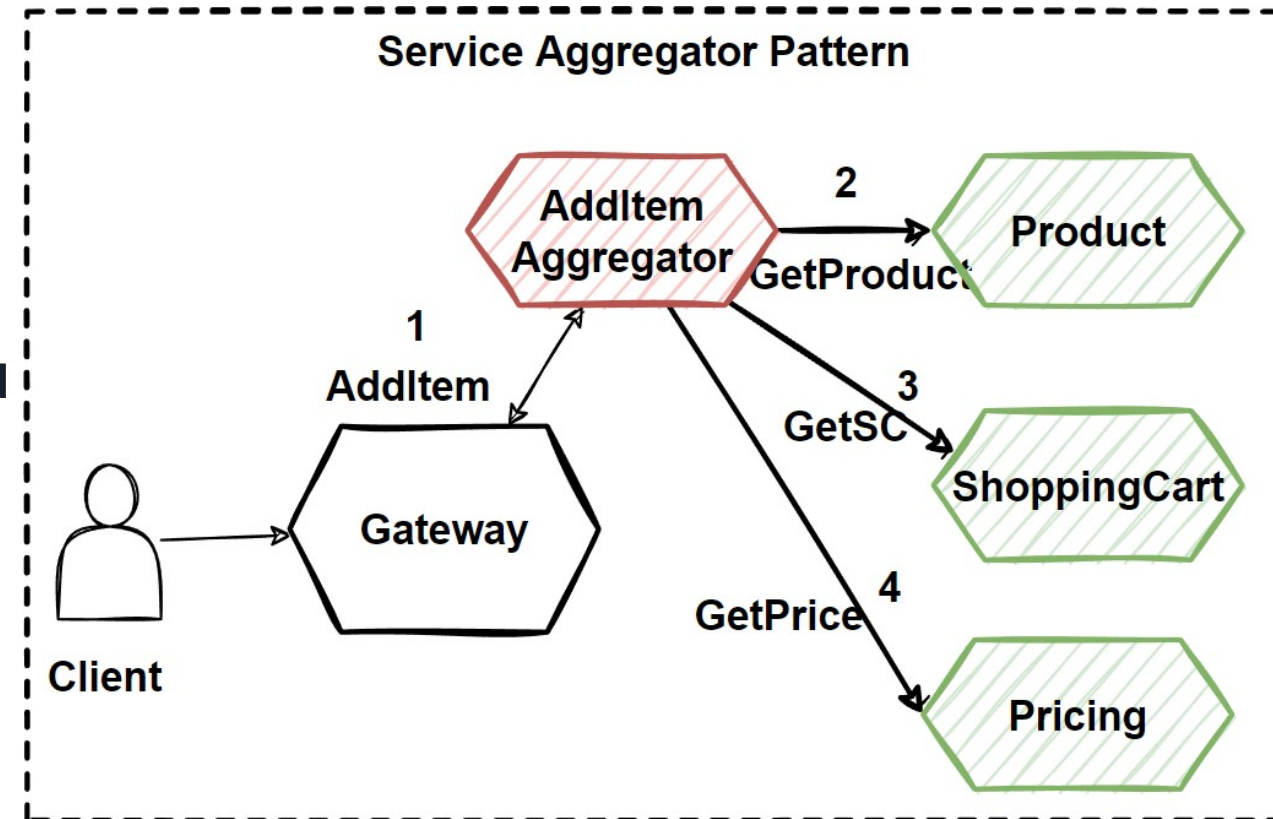
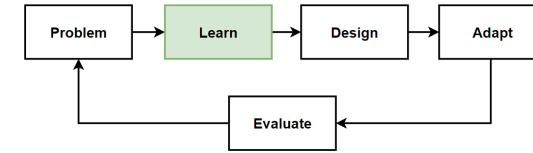
Solutions

- Aggregate query operations
- **Service Aggregator Pattern**



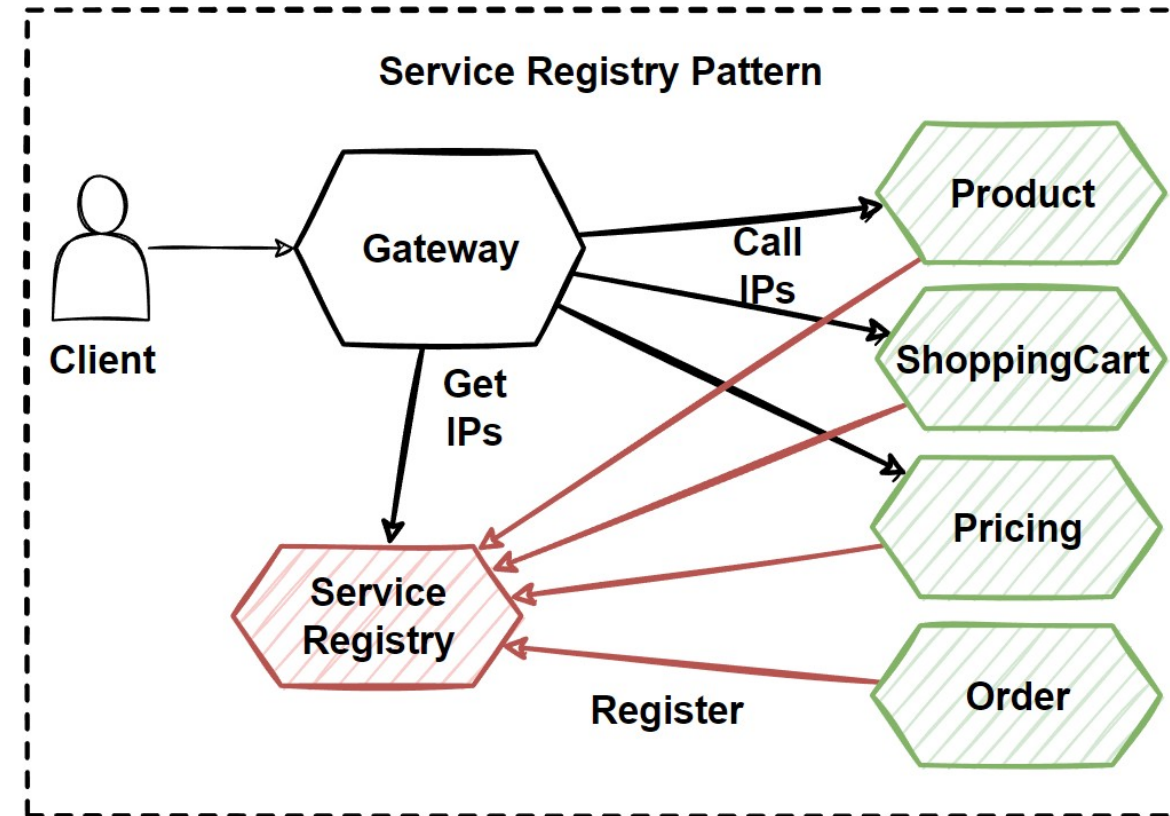
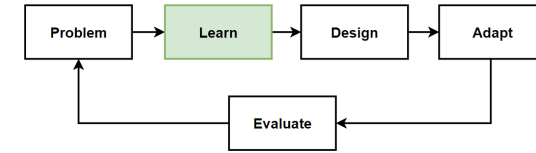
Service Aggregator Pattern

- **Service Aggregator Pattern** receives a request from the client or API Gateway.
- **Dispatches requests** of multiple internal backend microservices.
- **Combines the results** and responds back to the initiating request in 1 response structure.
- **Reduces chattiness** and **communication overhead** between the client and microservices
- AddItem **aggregates request data** from to several back-end microservices: Product - SC and Pricing.
- **Isolates the underlying addItem operation** that makes calls to multiple back-end microservices.
- **Centralizing its logic** into a specialized microservice.



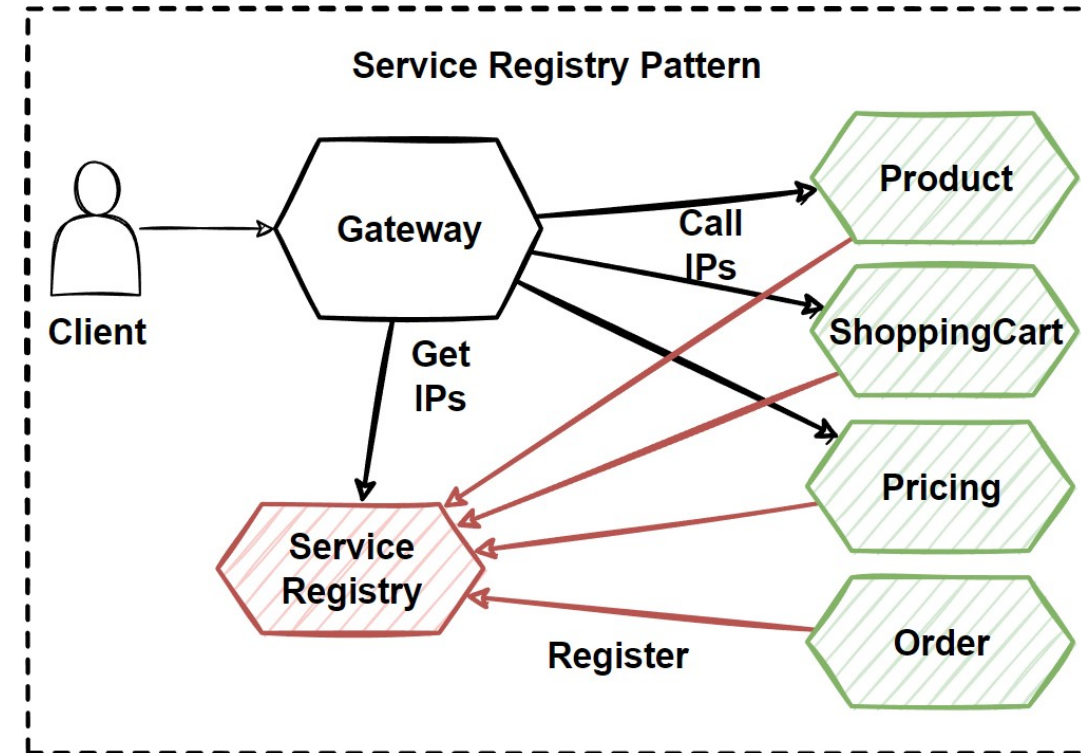
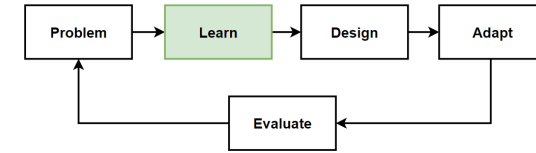
Service Registry/Discovery Pattern

- Microservice **Discovery Patterns** and **Service Registry** provide to register and discover microservices in the cluster.
- **Why We Need Service Discovery Pattern ?**
When deploying and scaling microservices, more microservices and instances could be added to the system to provide scalability to the distributed application.
- **Service locations** can **change frequently**, more dynamic configuration is needed for the microservice architecture.
- **IP Addresses** and **network locations** are **dynamically** assigned and often change due to auto-scaling features.
- **Service Registry** and **Discovery pattern** allows to find the network locations of microservices without injecting or coupling services.



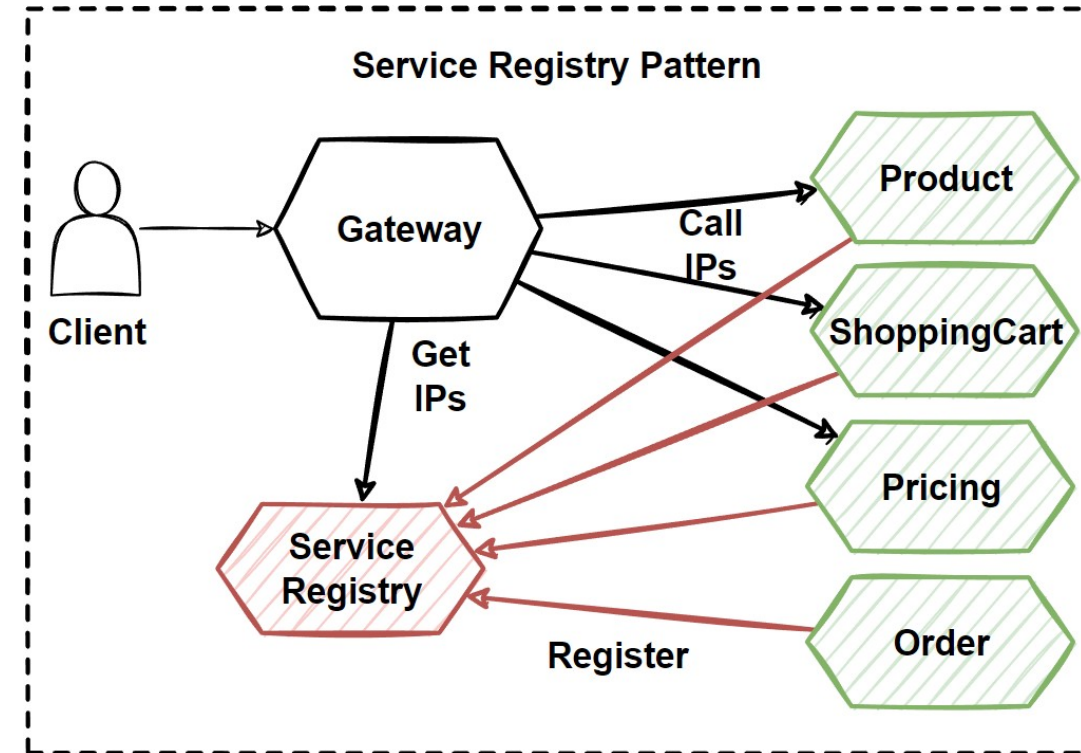
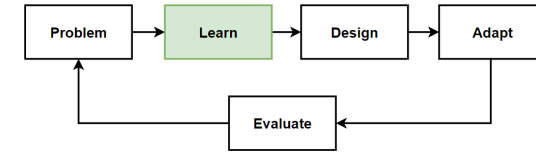
What is Service Discovery Pattern?

- To find the **network locations** of microservices, the **service discovery pattern** is used in microservices applications.
- It will provide to **register** and **discover microservices** in the cluster.
- **API Gateways** for **routing the traffic** with client and internal microservices. How API Gateways access the internal backend microservices ?
- **Service discovery pattern** uses a **centralized server** for «service registry» to maintain a central view of microservices network locations.
- Services **update their locations** in the **service registry** at fixed intervals. Clients can connect to the service registry and fetch the locations of microservices.
- There are **2 main service discovery patterns**:
 - Client-side service discovery
 - Server-side service discovery



How Does Service Discovery Work?

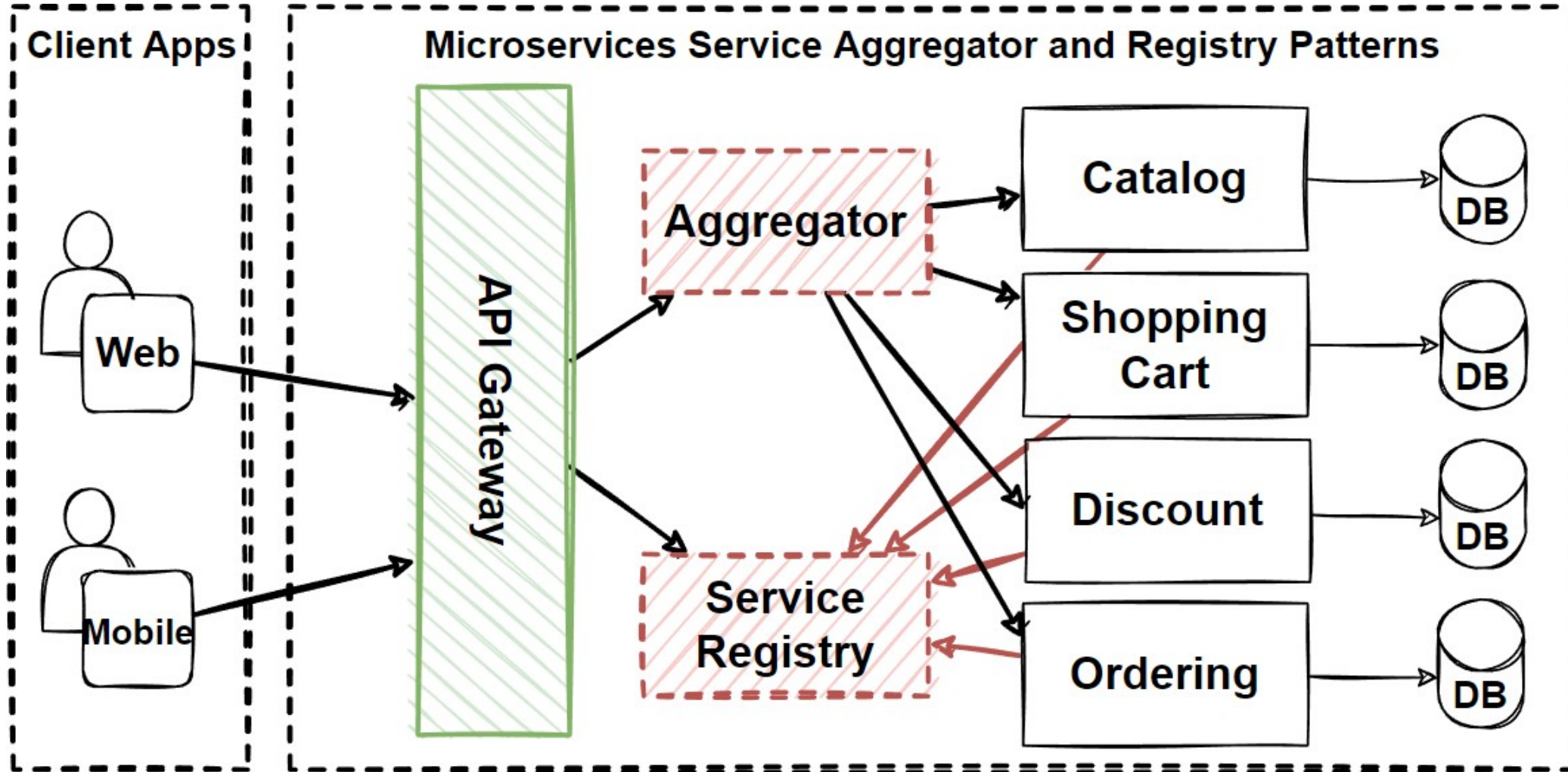
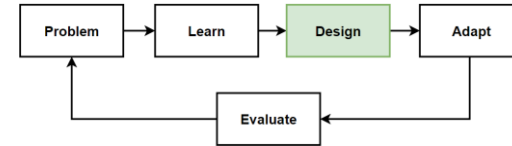
- **3 participants:** Service Registry, Client, Microservices
- **Client** service finds the location of other service instances by **querying a Service Registry**.
- Internal backend services **register to Service Registry** service before. The Service Registry is a **database for microservice instances**.
- The service registry **keeps records of the network locations** of the microservices.
- When the client service need to access internal services, then it will **query from Service Registry** and access them.
- Clients can **find the locations** of microservices using the **service registry** and directly call them.
- **Netflix** provides a service discovery pattern called «**Netflix Eureka**»
- **Kubernetes** container orchestrator **automatically handle** to service discovery operations.



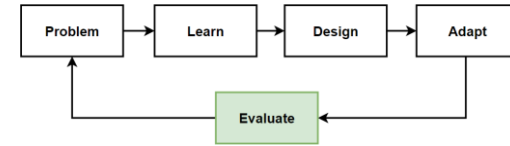
Before Design – What we have in our design toolbox ?

Architectures	Patterns&Principles	Microservices Communications	Non-FR	FR
• Microservices Architecture	• The Database-per-Service Pattern • Polygot Persistence • Decompose services by scalability • The Scale Cube • Microservices Decomposition Pattern • Microservices Communications Patterns	• HTTP Based RESTful API • GraphQL API • gRPC API • WebSocket API • Gateway Routing Pattern • Gateway Aggregation Pattern • Gateway Offloading Pattern • API Gateway Pattern • Backends for Frontends Pattern-BFF • Service Aggregator Pattern • Service Registry/Discovery Pattern	• High Scalability • High Availability • Millions of Concurrent User • Independent Deployable • Technology agnostic • Data isolation • Resilience and Fault isolation	• List products • Filter products as per brand and categories • Put products into the shopping cart • Apply coupon for discounts • Checkout the shopping cart and create an order • List my old orders and order items history

Design: Microservices Architecture with Service Aggregator, Registry/Discovery Pattern



Evaluate: Microservice Architecture with Service Aggregator, Registry/Discovery Pattern



Benefits

Aggregator

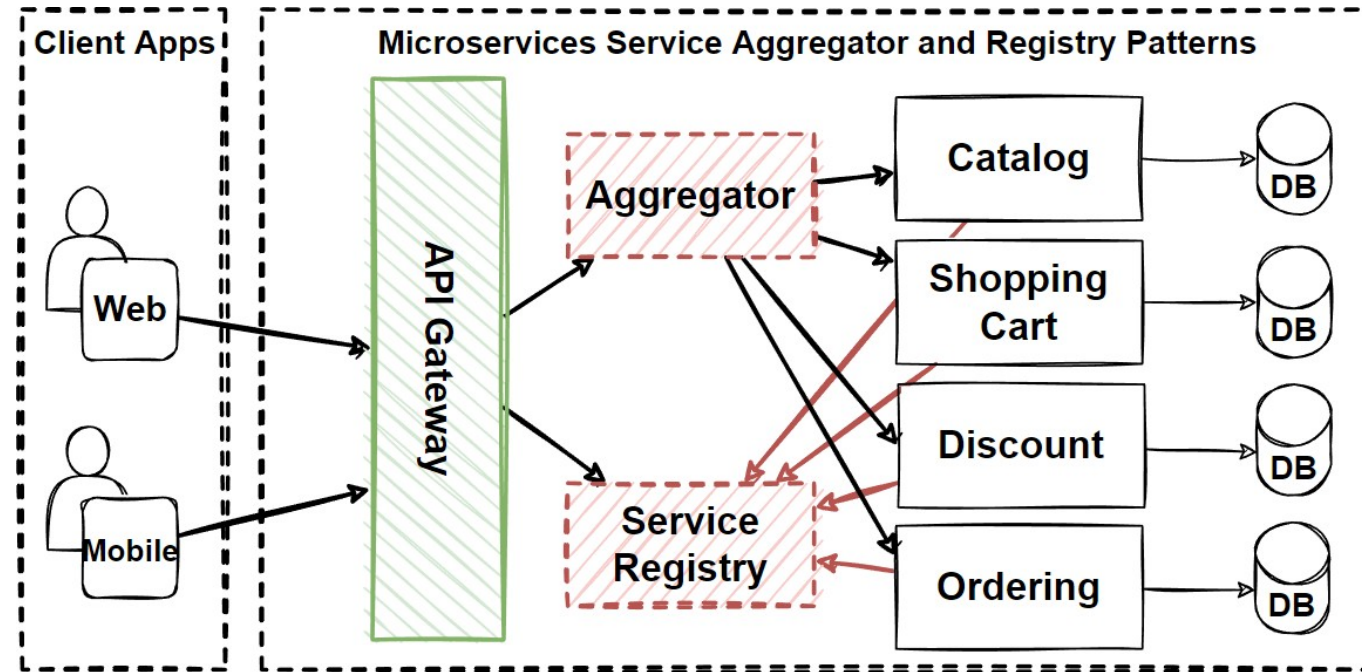
- Reduces chattiness and communication overhead
- Abstract operations that calls to multiple back-end microservices
- Centralizing logic into a specialized microservice

Registry/Discovery

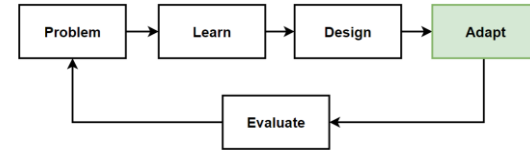
- Provide to register and discover microservices in the cluster
- Keep track of microservice locations beforehand

Drawbacks

- Increased Latency
- Good for large-scaled microservices application which has several client applications
- Only for Sync communications problems



Adapt: Microservice Architecture with Service Aggregator, Registry/Discovery Pattern



Frontend SPAs

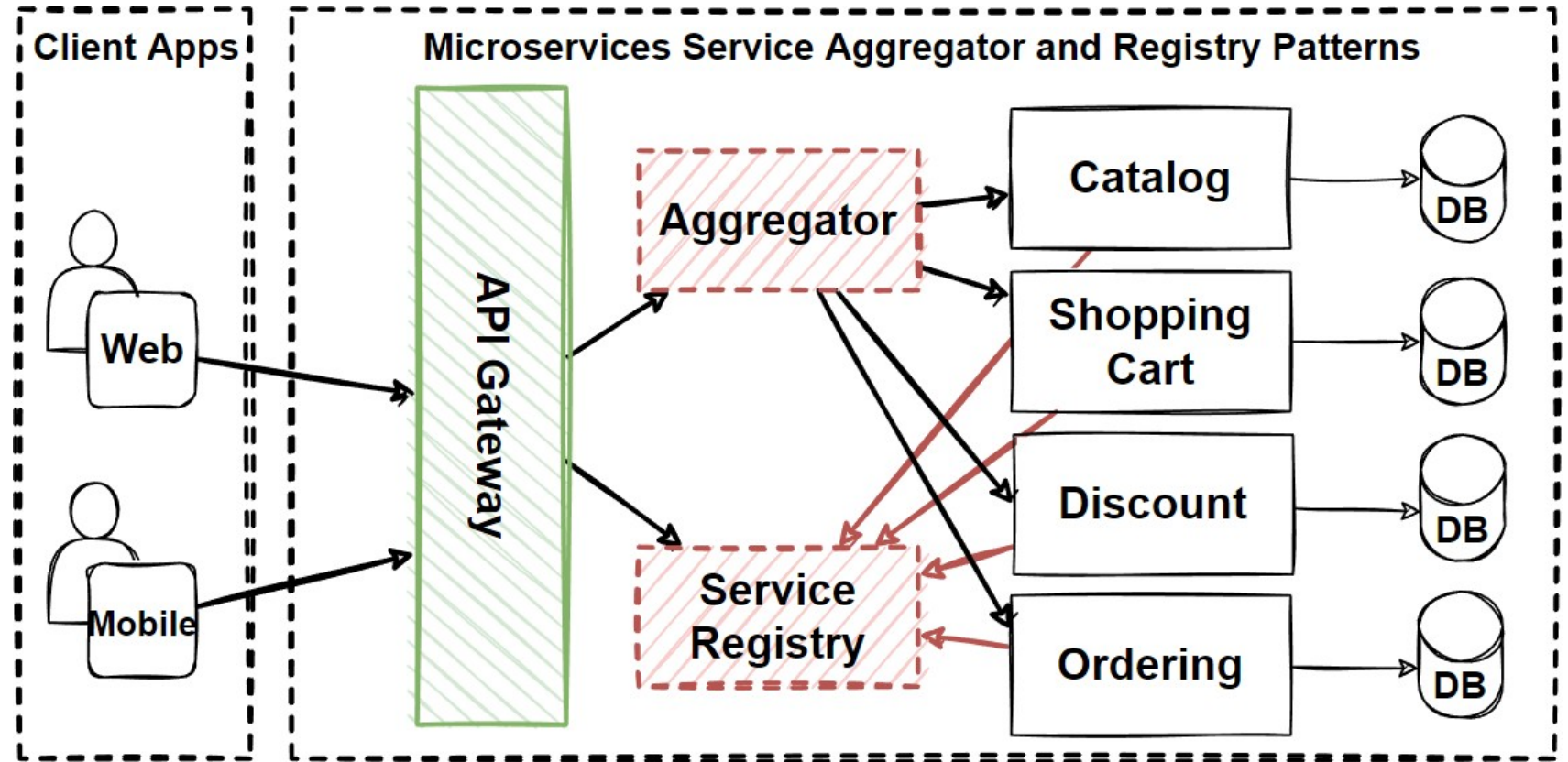
- Angular
- Vue
- React

API Gateways

- Kong Gateway
- Tyk API Gateway
- Express Gateway
- Amazon AWS API Gateway

Service Registry

- Netflix Eureka
- Kubernetes
- Serverless Orchestrators



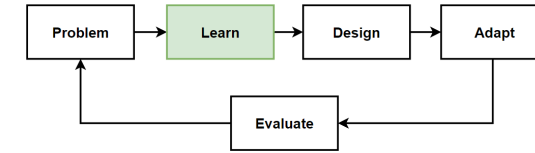
Backend Microservices

- Java – Spring Boot
- .Net – Asp.net
- JS – NodeJS

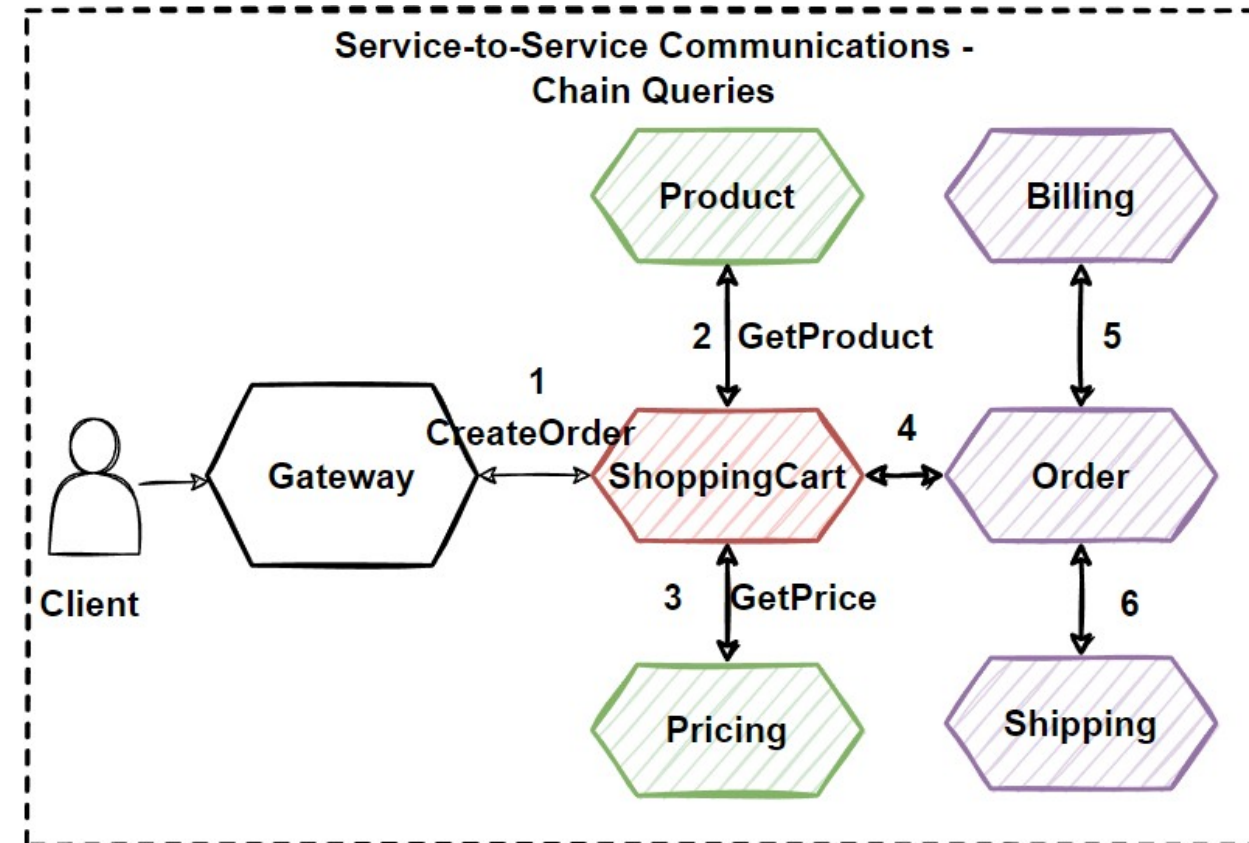
Database

- MongoDB – NoSQL Document DB
- Redis – NoSQL Key-Value Pair
- Postgres – Relational

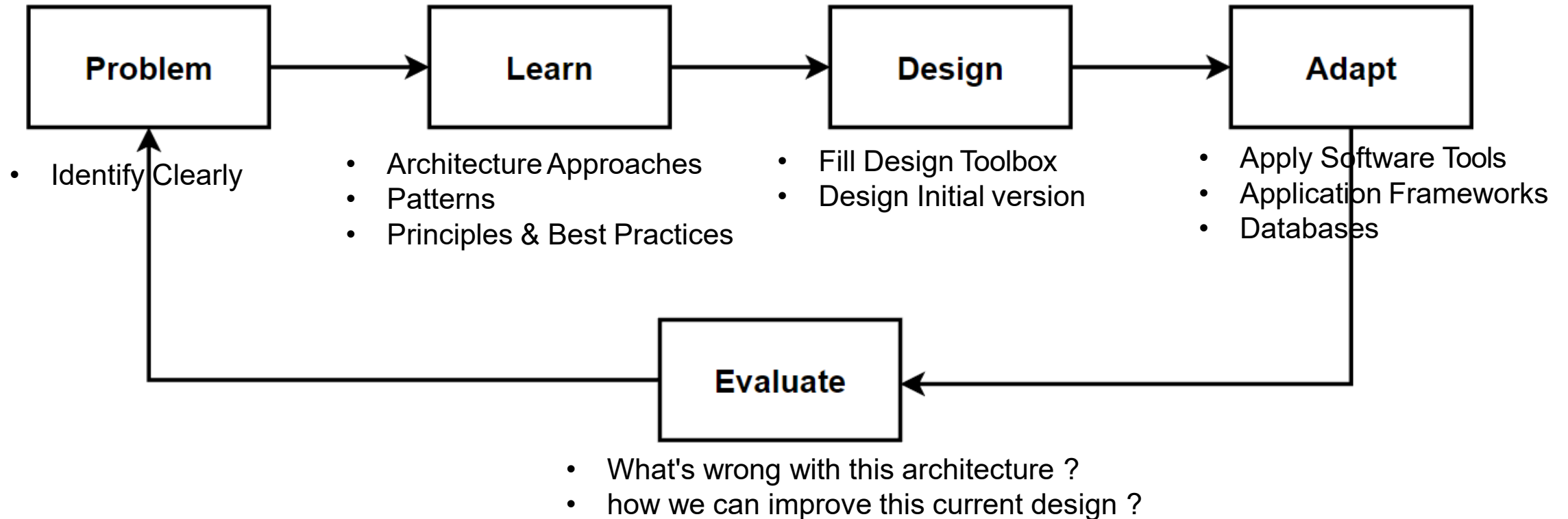
Service-to-Service Communications Chain Queries



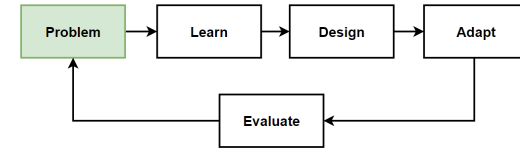
- If **service calls** are much then a few HTTP calls to multiple microservices, than it goes to **un-manageable** situation.
- **Place Order Use Case:** client checkout shopping cart, start order fulfillment processes.
- Request/Response Sync Messaging end up **with 6 sync chain HTTP Request.**
- **Increase latency** and **negatively impact** the performance, scalability, and availability.
- What if the **step 5 or 6 is failed ?**
- We can **apply 2 way** to solve this issues:
 - **1- Change microservices communications to async way**
 - 2- Use Service Aggregator Pattern to aggregate somequery operations in 1 API Gateway.



Way of Learning – The Course Flow



Problem: Long Running Operations Can't Handle with Sync Communication



Problems

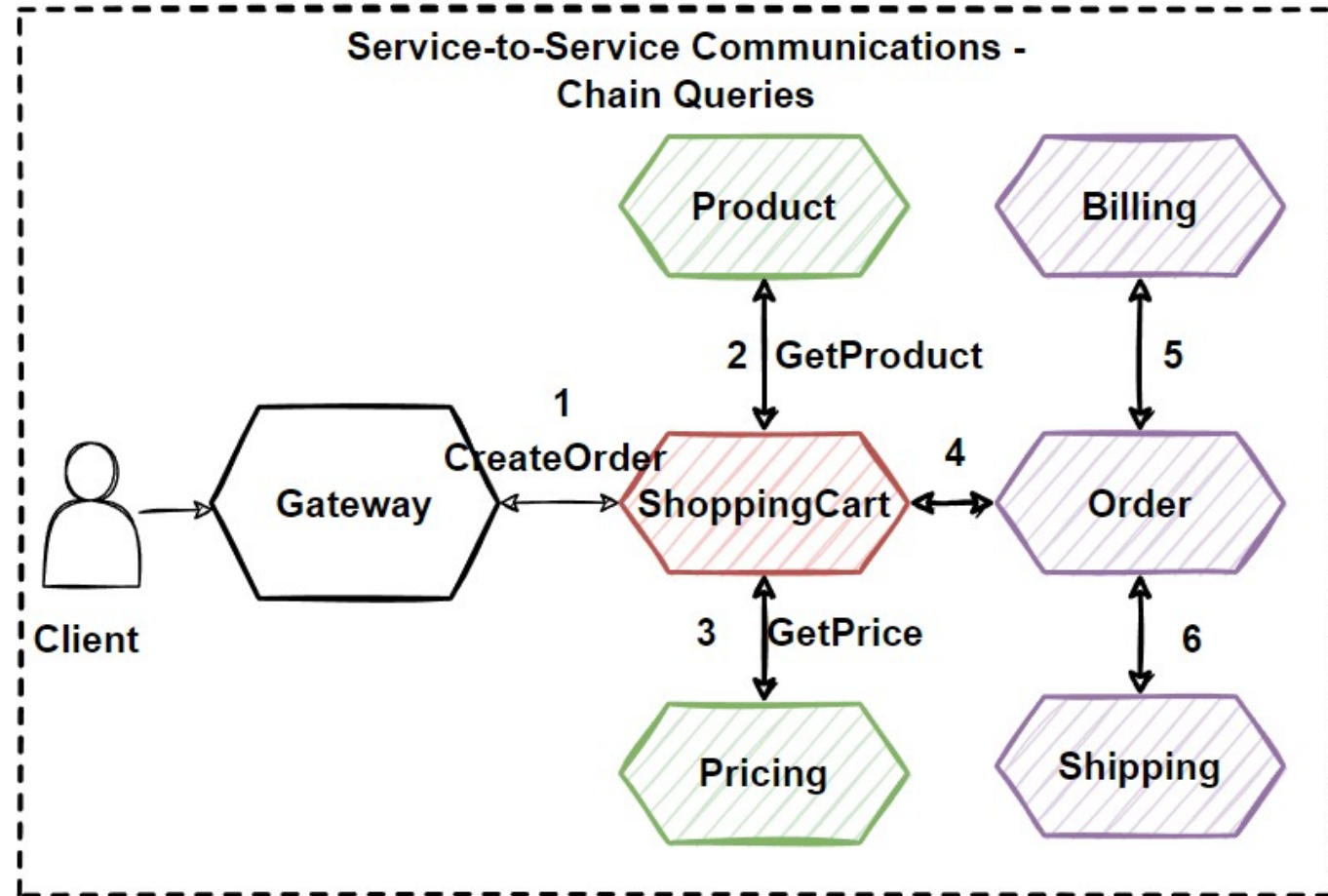
- HTTP calls to multiple microservices
- Chain Queries
- Visit more than a few microservices
- Increased latency with Highly Coupling Services
- Performance, scalability, and availability problems

Best Practices

- Minimize the communication between the internal microservices
- Make microservices communication in Asynchronous way as soon as possible.

Solutions

- **Asynchronous Message-Based Communications**
- Working with events



**CÁM ƠN ĐÃ CHÚ Ý
LẮNG NGHE!**